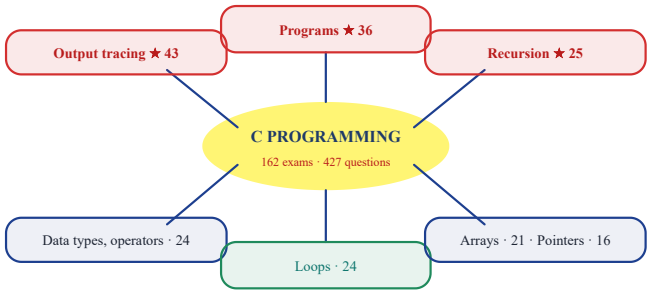
Structured Programming (C)

every C / structured-programming question asked in BCS · NTRCA · BPSC · bank · power-sector · BUET-set papers, solved in 41 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
> ✎ **Asked in:** the exams it was taken from, e.g. BPSC AP 2023 · 45th BCS CSE (2024)

Every program and every "find the output" answer was compiled and run; green = the real output. Where C leaves the result **undefined**, the sheet says so.

Most asked
① call by value vs reference · 5
② recursive factorial · 5
③ compiler vs interpreter · 5 + 3
④ array vs structure · 4
⑤ Fibonacci · local vs global · ++i vs i++ · 2 each

Syllabus coverage

"**Structured programming**: algorithms, flowcharts, data types, operators, **control structures**, functions, **recursion**, arrays, strings, **pointers**, structures and unions, file I/O", in C.

> **Legend:** **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = checked answer

Contents

- ✓ Languages · translators · errors … p. 2–4
- ✓ Algorithm, flowchart, GCD … p. 5–7
- ✓ Structured · data types · operators p. 8–10
- ✓ Loops · decision programs … p. 11–14
- ✓ Series · digit · number programs p. 15–20
- ✓ Functions · call by reference … p. 21–22
- ✓ Recursion …… p. 23–24
- ✓ Arrays · matrices …… p. 25–28
- ✓ Pointers …… p. 29–30
- ✓ Strings …… p. 31–32
- ✓ Structure, union · files … p. 33–35
- ✓ Find the output I–V …… p. 36–40
- ✓ Night-before · practice MCQs … p. 41

1 Programming languages: low, high, "middle"

> **Q1:** Difference between high-level and low-level languages, with examples. / How many kinds of programming language are there? Write the advantages and disadvantages of high-level languages. / Write the names of some high-level languages.
> ✎ **Asked in:** SPCBL SAE 2022 · 15th NTRCA Computer Operation (2019) · SPARRSO TO(IT) 2021

	Machine language	Assembly language	High-level language
written in	0 and 1	mnemonics: `MOV AX, 5`	English-like: `x = a + b;`
translator	**none** (runs directly)	assembler	compiler / interpreter
speed	**fastest**	fast	slower (after translation)
portable?	no: one CPU	no	**yes**

Generations: 1GL machine, 2GL assembly, 3GL procedural (C, Pascal), 4GL (SQL), 5GL (Prolog). High-level **pros**: easy, portable, fewer errors, libraries. **Cons**: needs translation, slower, little hardware control. Names: **FORTRAN, COBOL, C, C++, Java, Python, BASIC**.

MCQ as asked	answer	asked in
Which is a high-level programming language?	**C / C++ / Java**	BB AD(General) 2022
Which is high-level? A) Machine code B) C++ C) Assembly D) C	**B and D**	BB AD(General) 2026
Understood without translating / needs no translation	**machine language**	BPSC AP 2022 · Combined Bank Officer(IT) 2020
The first high-level programming language	**FORTRAN** (1957)	Combined Bank Officer(IT) 2020
A programming language requires users to follow rules of ___	**syntax**	Combined Bank Officer(IT) 2020
Object code is the ___ language file that tells the CPU what to do	**machine**	Combined Bank Officer(IT) 2020
___ is natural-language statements that look like programming code	**pseudocode**	Combined Bank Officer(IT) 2020
Making object code of one system work on another	**porting**	Combined Bank Officer(IT) 2020
What kind of language is C? low / mid / high level / none	**see trap**	DoICT AP 2020

2 Compiler vs interpreter, linker, loader, JIT

Q2: Distinguish between compiler and interpreter, with examples. / Write the differences between compiler and interpreter.
✎ **Asked in:** 40th BCS (2020) · BCC Programmer 2019 · BPSC ADA 2022 · BPSC AP 2022 · PGCB SAE 2021 · 15th NTRCA Computer Operation (2019) · 16th NTRCA CSE (2019) · SPARRSO TO(IT) 2021 · Combined Bank AP(SB & Janata) 2026 · ICB AMCL AP 2024

Point	Compiler	Interpreter
translates	the **whole program** at once	**line by line**, while running
output	an object / executable file	no object file
errors	all listed after compiling	stops at the first error
execution speed	**fast**	slow (translates every run)
memory	more (object code)	less
source needed to run?	no	yes
examples	**C, C++, Pascal** (gcc, Turbo C)	**Python, BASIC, JavaScript**

Q3: What is a translator program? Write its functions / its types.
✎ **Asked in:** 17th NTRCA School (2023) · 17th NTRCA CSE (2023) · 15th NTRCA Computer Operation (2019)

A **translator** converts source code to machine code; it also detects and reports syntax errors. Types: **assembler** (assembly → machine), **compiler**, **interpreter**.

MCQ as asked	answer	asked in
Converts an entire program into machine language: interpreter / converter / simulator / compiler	**compiler**	BPSC AP 2022
A ___ translates a file of source code into machine language	**compiler**	Combined Bank Officer(IT) 2020
The system software that creates the final executable file	**linker**	Teletalk Executive 2019

Q4: What is the LEGv8 architecture? An 8086 has code segment 4000H and instruction pointer 9F20H: find the memory address.
✎ **Asked in:** 45th BCS ICT (2024)

LEGv8 is the teaching subset of ARMv8 (64-bit RISC, 32 registers, load/store) from Patterson & Hennessy. 8086 physical address = segment × 16 + offset = 40000H + 9F20H = **49F20H**.

Exam trap: "Is C high- or mid-level?" C is a **high-level** language with low-level features (pointers, bit operators), so Bangladeshi keys usually expect **middle-level**; the source key printed "high level". Write "middle-level (high-level with low-level access)". In BB 2026 both **C++** and **C** are right.

3 From source to running program; kinds of error

Q5: What are the tasks of the linker and the loader? Describe with examples. / Write down the role of a linker.
✎ **Asked in:** BPSC AP 2023 · BUET M.Sc. May 2016



Linker: joins the object files and library code into one executable and resolves external references (the call to `printf` is matched to its code in the C library). **Loader**: copies the executable into memory, sets addresses (relocation) and starts it. Errors: "undefined reference to `foo`" is a **linker** error.

- **JIT compiler**: compiles bytecode to native code at run time, method by method, and caches it; the same bytecode runs on any machine that has a JVM, so the program stays machine-independent while running fast. ✎ 46th BCS CSE
- **Comments**: the preprocessor removes every comment, replacing it with one space, before compiling: no code, no size, no speed cost. ✎ BPSC AP 2023
- **Processor per language**: C/C++ compiler; Java compiler + JVM (interpreter + JIT); Python interpreter. ✎ BUET M.Sc. May 2016
- **Platform-independent software** runs unchanged on any OS/CPU, e.g. a Java program (bytecode on any JVM), a Python script, a web app. ✎ Combined Bank SO(IT) 2023

> **Q6:** Explain the errors made during coding, with examples. / Difference between compile-time and run-time errors.
> ✎ **Asked in:** 16th NTRCA CSE (2019) · RAKUB ANSE 2026

Error	When / who finds it	Example
syntax (compile time)	compiler, before running	`printf("hi")` without `;`
semantic / type	compiler	`int x = "abc";`
linker	linker	`main` missing, undefined function
run-time	while running: program crashes	divide by zero, null pointer
logical	nobody: wrong output	`avg = a + b / 2;`

- Error generated when the rules of a language are violated → **syntax error**. ✎ Teletalk Executive 2019
- Error generated while the program is being executed → **run-time error**. ✎ Combined Bank Officer(IT) 2020

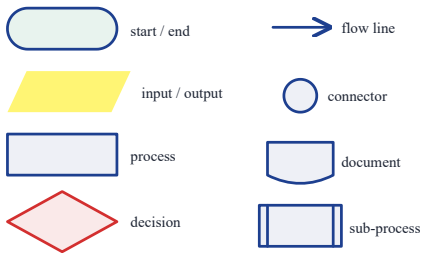
> **Memory trick:** The **compiler** finds what you wrote wrong, the **linker** finds what is missing, **run time** finds what goes wrong, and **nobody** finds a logical error — test with sample input.

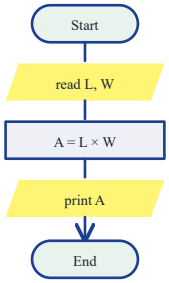
4 Algorithm, flowchart, program development, errors

> **Q7:** What is an algorithm? Write its characteristics. / Write the steps of program development. / What is pseudocode?
> ✎ **Asked in:** 17th NTRCA School (2023) · 15th NTRCA CSE · 17th NTRCA CSE (2023)

An **algorithm** is a finite, ordered set of unambiguous steps that solves a problem. Characteristics: **input** (zero or more), **output** (at least one), **definiteness**, **finiteness**, **effectiveness**. **Pseudocode** writes the algorithm in plain English-like statements that look like code. Program development: ① define the problem ② analyse ③ design (algorithm, flowchart) ④ code ⑤ test and debug ⑥ document ⑦ maintain.

> **Q8:** Write the flowchart symbols with their meanings. / Draw a flowchart to find the area of a rectangle.
> ✎ **Asked in:** 17th NTRCA School (2023)





> **Q9:** Draw a flowchart to display the numbers 1 to 100.
> ✎ **Asked in:** BTCL JAM 2024

Start → i = 1 → **decision i ≤ 100?** → yes: print i, **i = i + 1**, back to the decision → no: End.

> **Exam trap:** The printed chart uses i = i + 2 and "i < 100": that prints only the odd numbers 1–99.

> **Q10:** Draw a flowchart for an ATM PIN check: user input, decision success / failure.
> ✎ **Asked in:** Sonali Bank ADA 2026

Start → tries = 0 → read PIN → **PIN correct?** yes → "Success", show menu → End. no → tries = tries + 1 → **tries < 3?** yes → back to read PIN; no → "Failure: card blocked" → End.

> **Memory trick:** An algorithm's five properties spell **FIDEO**: **F**initeness, **I**nput, **D**efiniteness, **E**ffectiveness, **O**utput.

5 Largest of three: algorithm, flowchart, program

Q11: Write an algorithm, draw a flowchart and write a C program to find the largest of three numbers. / What are algorithm and flowchart? Write an algorithm for the largest of three. / What is pseudocode? Draw a flowchart for the largest of three. / Draw or describe a flowchart to determine the largest of three distinct numbers A, B, C. / Find the maximum of three given numbers.

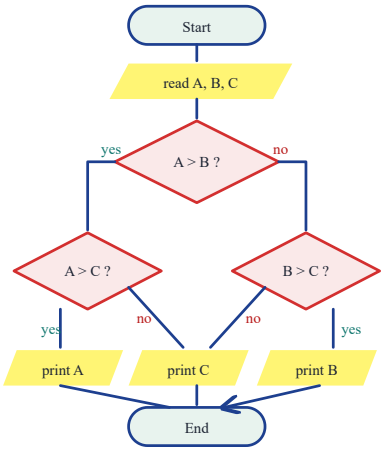
✎ **Asked in:** 13th NTRCA CSE (2016) · 15th NTRCA Computer Operation (2019) · BCPCL JI 2019 · 17th NTRCA CSE (2023) · BTCL JAM 2024 · Combined Bank AP 2026 · Bank (name cut off) AP · DESCO AE(CSE) 2016

Algorithm (pseudocode):

1. Start; read A, B, C.
2. MAX ← A.
3. If B > MAX then MAX ← B.
4. If C > MAX then MAX ← C.
5. Print MAX; stop.

```
int a, b, c, max;
scanf("%d %d %d", &a, &b, &c);
max = a;
if (b > max) max = b;
if (c > max) max = c;
printf("Largest = %d\n", max);
```

input 83 40 3 → Largest = 83



the flowchart the examiners expect

Q12: Write an algorithm and a C program to find the smallest of three numbers. / Flowchart for the smallest of three.

✎ **Asked in:** 18th NTRCA School-2 (2024) · BTCL JAM 2024

```
int a, b, c, min;
scanf("%d %d %d", &a, &b, &c);
min = a;
if (b < min) min = b;
if (c < min) min = c;
printf("Smallest = %d\n", min);
```

Same algorithm and chart with every > turned into <. input 83 40 3 → Smallest = 3

Q13: Write the code for the second highest of three given numbers (83, 40, 3) in C/C++.

✎ **Asked in:** BOF AE(CSE) 2021

```
int a = 83, b = 40, c = 3, mid;
if ((a >= b && a <= c) || (a <= b && a >= c)) mid = a;
else if ((b >= a && b <= c) || (b <= a && b >= c)) mid = b;
else mid = c;
printf("Second highest = %d\n", mid);
```

→ Second highest = 40

Exam trap: The printed solution finds the **largest** (83), and its `Int`, `pirintf` do not compile.

6 GCD, LCM and factorial

Q14: Write an algorithm to find the GCD (গ.সা.গু) of two numbers. / Draw a flowchart and write the algorithm for the GCD of two numbers.

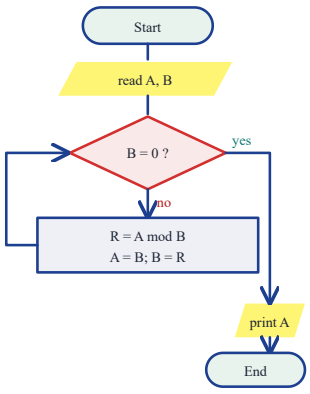
✎ **Asked in:** 17th NTRCA School (2023) · 18th NTRCA School (2024)

Euclid's algorithm: GCD(a, b) = GCD(b, a mod b); stop when b = 0.

1. Start; read A, B.
2. While B ≠ 0: R ← A mod B; A ← B; B ← R.
3. Print A (the GCD); stop.

```
int a, b, r;
scanf("%d %d", &a, &b);
while (b != 0) { r = a % b; a = b; b = r; }
printf("GCD = %d\n", a);
```

input 96 36 → GCD = 12 (96,36 → 36,24 → 24,12 → 12,0)



Q15: Given two integers A and B as input, write a program to compute the least common multiple (LCM) of A and B.
✎ **Asked in:** BB AP 2023

```
long long a, b, x, y, r;
scanf("%lld %lld", &a, &b);
x = a; y = b;
while (y != 0) { r = x % y; x = y; y = r; }   /* x = GCD */
printf("LCM = %lld\n", a / x * b);
```

LCM = a × b / GCD; divide first (`a / gcd * b`) so the product cannot overflow. input 12 18 → LCM = 36 (the same paper printed it twice, once as "LCN").

Q16: Write an algorithm, draw a flowchart and write a C program to find the factorial of an input number.
✎ **Asked in:** 15th NTRCA CSE

1. Start; read N.
2. F ← 1, I ← 1.
3. While I ≤ N: F ← F × I; I ← I + 1.
4. Print F; stop.

Flowchart: the GCD chart's loop with the decision "I ≤ N ?" and the process box "F = F × I; I = I + 1".

```
int n, i;  long long f = 1;
scanf("%d", &n);
for (i = 1; i <= n; i++) f = f * i;
printf("%d! = %lld\n", n, f);
```

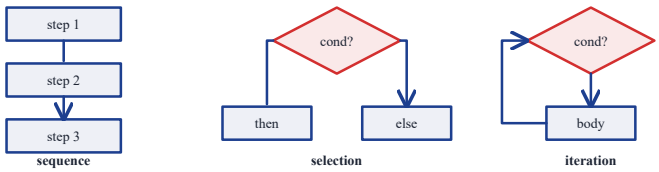
input 5 → 5! = 120

Memory trick: GCD loop = "**remainder, shift, repeat**": r = a % b; a = b; b = r. LCM = product ÷ GCD.

7 Structured programming and problem solving

Q17: What is structured programming / a structured programming language? Explain with an example. / What are the main objectives of structured programming? Write two advantages. / Define program, identifier and structured language.
✎ **Asked in:** 38th BCS (2018) · SPARRSO 2020 · 13th NTRCA CSE (2016)

Structured programming: writing a program from **three control structures only** — **sequence, selection (if/switch), iteration (loops)** — divided top-down into functions (modules) with one entry and one exit each, and no `goto`. Languages: **C, Pascal, ALGOL, Ada**.



Objectives: readable, reliable, easy to test and maintain code. **Advantages**: ① easy to read and debug, because control flows top to bottom; ② modules can be written, tested and reused separately. **Definitions**: a **program** is a set of instructions that tells the computer how to solve a problem; an **identifier** is the name of a variable, function or array (letters, digits, _; not starting with a digit; not a keyword).

Q18: Define problem-solving technique. Write the differences between structured programming and object-oriented programming. / What is programming? Give 4 examples. Difference between structured and OOP programming.
✎ **Asked in:** 45th BCS CSE (2024) · Sonali Bank ADA 2026

Problem-solving technique: a systematic method — understand the problem, divide it into sub-problems, design an algorithm (and flowchart), code, test, refine, document. **Programming** = writing such instructions in a language: e.g. **C, C++, Java, Python**.

	Structured	Object-oriented
built from	functions	classes and objects
design	**top-down**	bottom-up
data	global, open to every function	hidden inside objects
reuse	copy functions	inheritance
examples	C, Pascal	C++, Java

Memory trick: Structured = "**S-S-I**": Sequence, Selection, Iteration — and nothing else.

8 Data types, identifiers, local vs global

Q19: Describe the different data types used in C.
✎ **Asked in:** 18th NTRCA College ICT (2024)

type	size*	range / use	format
`char`	1	−128 … 127, one character	`%c`
`int`	4 †	about ±2.1 × 10^9	`%d`
`short` · `long long`	2 · 8	±32 767 · ±9.2 × 10^{18}	`%hd` · `%lld`
`float` · `double`	4 · 8	≈ 7 · 15 significant digits	`%f` · `%lf`
`void`	—	"no value"	

* bytes by `sizeof` on a 64-bit compiler; † 2 on 16-bit Turbo C. Groups: **basic** (above), **derived** (array, pointer, function), **user-defined** (struct, union, enum, typedef).

Q20: Can you store the value 32678 in a variable of type `int`? If not, why?
✎ **Asked in:** 15th NTRCA · 15th NTRCA ICT

On a **16-bit** compiler (Turbo C) `int` is 2 bytes: range −32 768 … **32 767**, so 32 678 does **not** fit (it wraps to −32 858). Use `long` or `unsigned int`. On today's 32/64-bit compilers `int` is 4 bytes, so it fits — write both.

- Invalid variable name: Average / **No#of-students** / Xzy / y23z → `#` and `-` are not allowed. ✎ DoICT ANE 2019 · DoICT AP 2020
- To retain a variable's value between function calls use **static**. ✎ 49th BCS CSE (2025)
- `int x = 5, y = 10, *p = &x; y = *p; y++;` — x before return: **(d) 5** (only the copy in y changes). ✎ BUET M.Sc. April 2021

Q21: What is a variable? Difference between local and global variables.
✎ **Asked in:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 17th NTRCA

Local	Global
inside a function / block	outside all functions
seen in that block	seen everywhere
lives while it runs	whole program
garbage at start	0 at start

```c
#include <stdio.h>
int count = 0;              /* global */
void hit(void) { int n = 1; count += n; }   /* n: local */
int main(void) {
    hit(); hit();
    printf("count = %d\n", count);
    return 0;
}
```

→ count = 2

A **variable** is a named memory location whose value can change. **Null vs void**: `NULL` is a pointer value pointing nowhere; `void` is a type meaning "no value" (`void f()`) or "any type" (`void *`). ✎ **Asked in:** BCC AP 2023

- **Type conversion**: implicit (`int` → `double` in `2 + 1.5`) or explicit cast `(int)3.9` = 3. Side effects: loss of fraction, overflow, sign change (−1 → 4294967295 as unsigned), precision loss. ✎ 13th NTRCA CSE (2016)
- **Function prototype** = declaration before use: `int sum(int, int);`. **Overloaded functions** share a name with different parameters: C++ only, not C. ✎ 15th NTRCA CSE

9 Increment, sizeof, precedence, ternary

Q22: What are increment and decrement operators? Difference between ++i and i++ (++a and a++) with examples. / What is nested if?
✎ **Asked in:** 17th NTRCA ICT (2023) · NU AP 2020 · 15th NTRCA · 15th NTRCA ICT · 16th NTRCA CSE (2019)

`++i` (prefix)	`i++` (postfix)
increment first, then use	**use first**, then increment
`b = ++i;` b gets the new value	`a = i++;` a gets the old value

```c
int i = 5, a, b;
a = i++;     /* a = 5, i = 6 */
b = ++i;     /* i = 7, b = 7 */
printf("a=%d b=%d i=%d\n", a, b, i);
```

→ a=5 b=7 i=7

Alone on a line (`i++;` or `++i;`) both just add 1. **Nested if** = an `if` inside another `if`/`else`: `if (m >= 40) { if (m >= 80) grade = 'A'; else grade = 'P'; } else grade = 'F';`

Q23: What is the difference between `sizeof c+1` and `sizeof(c+1)`? / between `sizeof 'c'+1` and `sizeof('c'+1)`?
✎ **Asked in:** BPSC AP 2023

```
char c = 'x';
printf("%zu %zu\n", sizeof c + 1, sizeof(c + 1));
printf("%zu %zu\n", sizeof 'c' + 1, sizeof('c' + 1));
```

`sizeof` binds tighter than +: `sizeof c + 1` = (1) + 1 = **2**, while `c + 1` is promoted to `int` → **4**. In C a character constant `'c'` is an `int`: **5** and **4**. → 2 4 · 5 4

MCQ as asked	answer	asked in
Which operator has right-to-left associativity? `,` / `+` / `?:` / `=`	**= (and ?:)**	49th BCS ICT (2025)
`x = 2 + 3 * 7 % 4 / 2 - 3;` x = ?	**−1**	BPDB AE(CSE) 2021
`(x < 0) ? 10 : 15` with x = 5	**15**	NWPGCL JAM(ICT) 2022
The format `%i` is used for the ___ type too	**int**	BUET IICT April 2021

```
int x = 2 + 3 * 7 % 4 / 2 - 3;
printf("%d\n", x);
```

* / % left to right: 3*7 = 21, 21 % 4 = 1, 1 / 2 = **0** (integer division) → 2 + 0 − 3 = −1. → -1

Exam trap: In the associativity MCQ **both** `?:` and `=` are right-to-left (the printed note admits it): the key is =; write "= (also ?:)". Precedence, high → low: `() [] → ++ -- ! (unary) * / % + - << >> < > == != & ^ | && || ?: = ,`

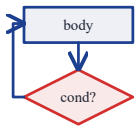
10 if-else, loops, break and continue

Q24: Write the format of the if…else statement. / What is looping? Why use it? Give an example. / Basic difference between looping and branching.
✎ **Asked in:** BPSC ANE 2020 · BCPSC AP 2018

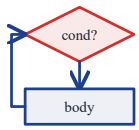
`if (cond) { … } else if (cond2) { … } else { … }`. A **loop** repeats a block while a condition holds (sum 1 to 100 without 100 lines). **Branching** picks *one path* once (if, switch); **looping** *repeats* it.

Q25: What is a loop? Write the structure, syntax and flowchart of for, while and do-while loops. / Draw the flowchart of a do-while loop. / Explain the structure of the FOR loop. / Difference between while and do-while.
✎ **Asked in:** 16th NTRCA · 15th NTRCA CSE · 16th NTRCA ICT · Cadet College Lecturer(ICT) 2025 · ICB AMCL AP 2024

loop	syntax
for	`for (init; cond; update) { … }`
while	`while (cond) { … }`
do-while	`do { … } while (cond);`



do-while



while / for

`while`	`do-while`
entry-controlled: tests first	**exit-controlled**: tests after the body
body may run **0** times	body runs **at least once**
no semicolon after `while(…)`	ends with `while(…);`

Q26: Distinguish between break and continue in a loop, with suitable examples.
✎ **Asked in:** Parliament Secretariat SAE 2023

```
for (int i = 1; i <= 6; i++) {
    if (i == 3) continue;  /* skip 3 */
    if (i == 5) break;     /* stop at 5 */
    printf("%d ", i);
}
```

`break` **leaves** the loop (or switch); `continue` **skips the rest of this iteration** and goes to the next. → 1 2 4

MCQ as asked	answer	asked in
Exit-controlled loop? for / while / do while / none	**do while**	49th BCS CSE (2025)
What does `continue` do inside a loop?	**skip to next iteration**	49th BCS CSE (2025)
Keyword to come out of a loop: break / if / switch / continues	**break**	DoICT ANE 2019
`exit()` causes an exit from …	**the program**	BPSC AP 2022

11 Calculator, triangle, bills, tax

Q27: Write a program for a basic calculator (+, −, *, /) using a switch statement.
✎ **Asked in:** 36th BCS (2016)

```
double a, b;  char op;
scanf("%lf %c %lf", &a, &op, &b);
switch (op) {
    case '+': printf("%.2f\n", a + b); break;
    case '-': printf("%.2f\n", a - b); break;
    case '*': printf("%.2f\n", a * b); break;
    case '/': if (b != 0) printf("%.2f\n", a / b);
              else printf("Division by zero\n");
              break;
    default:  printf("Invalid operator\n");
}
```

input 8 * 5 → 40.00

Every `case` ends in **break**, or control falls through to the next case; `default` catches a wrong operator.

Q28: Write a program to find the area of a triangle using IF-ELSE.
✎ **Asked in:** BCPCL JI 2019

```
double a, b, c, s;
scanf("%lf %lf %lf", &a, &b, &c);
if (a + b > c && b + c > a && a + c > b) {
    s = (a + b + c) / 2;                    /* Heron */
    printf("Area = %.2f\n", sqrt(s * (s - a) * (s - b) * (s - c)));
} else
    printf("Not a triangle\n");
```

input 3 4 5 → Area = 6.00

Q29: Write a Java method to compute an electricity bill: 4.00 per unit up to 100 units, 5.50 for units 101–200, 6.00 for the remaining units.
✎ **Asked in:** BPDB AE(CSE) 2021

```
public class Main {
    static double bill(int u) {
        if (u <= 100) return u * 4.00;
        if (u <= 200) return 100 * 4.00 + (u - 100) * 5.50;
        return 100 * 4.00 + 100 * 5.50 + (u - 200) * 6.00;
    }
    public static void main(String[] a) { System.out.println(bill(250)); }
}
```

bill(250) → 1250.0 (400 + 550 + 300)

Q30: An employee's weekly pay = hourly wage × regular hours + overtime hours × 1.5 × hourly wage. Read the wage, regular hours and overtime hours and print the total. / Read a cricket player's runs, innings and not-outs and display his average.
✎ **Asked in:** Combined Bank Officer(IT) 2020 · JBL ASA 2021

```
double w, reg, ot;
scanf("%lf %lf %lf", &w, &reg, &ot);
printf("Pay = %.2f\n", w * reg + ot * 1.5 * w);
```

```
java.util.Scanner sc = new java.util.Scanner(System.in);
int runs = sc.nextInt(), inn = sc.nextInt(), no = sc.nextInt();
System.out.println("Average = " + (double) runs / (inn - no));
```

200 40 5 → Pay = 9500.00 · 450 runs, 12 innings, 2 not out → Average = 45.0 (batting average divides by the times **out**)

12 Slab tax and a database login

Q31: Tax slabs: 0–2,50,000 → 0 %; 2,50,001–5,00,000 → 10 %; 5,00,001–10,00,000 → 20 %; above 10,00,000 → 30 %. (a) Write a program to calculate an employee's total tax. (b) From three employees' salaries find the highest tax payer.
✎ **Asked in:** NWPGCL AM(ICT) 2024

```
/* double tax(double s): slab by slab, top slab first (see note) */
double s[3], best = -1;  int who = 0;
for (int i = 0; i < 3; i++) {
    scanf("%lf", &s[i]);
    printf("Employee %d: tax %.0f\n", i + 1, tax(s[i]));
    if (tax(s[i]) > best) { best = tax(s[i]); who = i + 1; }
}
printf("Highest tax: employee %d\n", who);
```

`tax()`: `if (s > 1000000) { t += (s-1000000)*0.30; s = 1000000; }` then the same for 5,00,000 at 0.20 and 2,50,000 at 0.10. **Assumption**: the printed slabs are garbled (5000000, 100000); read as the slabs above, taxed slab by slab. → 5000 · 85000 · 185000 · employee 3

Q32: Write a program for a client that uses a database: the client logs in to a page with ID and password. / What is JDBC? Explain the steps to connect a Java application to a MySQL database.

✎ **Asked in:** Combined Bank ADA 2024 · Combined Bank Officer(IT) 2026

JDBC (Java Database Connectivity) is Java's standard API for SQL databases. Steps: ① add the MySQL driver (Connector/J) ② `DriverManager.getConnection(url, user, pass)` ③ create a `PreparedStatement` ④ execute and read the `ResultSet` ⑤ close (try-with-resources).

```
import java.sql.*;
public class Main {
    static boolean login(String id, String pass) throws SQLException {
        String url = "jdbc:mysql://localhost:3306/bank";
        String sql = "SELECT 1 FROM users WHERE user_id = ? AND password_hash = SHA2(?, 256)";
        try (Connection c = DriverManager.getConnection(url, "app", "secret");
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, id);  ps.setString(2, pass);   /* ? placeholders: no SQL injection */
            try (ResultSet rs = ps.executeQuery()) { return rs.next(); }
        }
    }
}
```

Compiles as shown (it needs a MySQL server to run). Marks: **PreparedStatement** with **?** placeholders (never string-concatenate the password into SQL), store a **hash** not the password, close the connection.

13 Printing patterns

Q33: Write a C++ program to print the pattern 1 2 3 4 5 / 1 2 3 4 / 1 2 3 / 1 2 / 1. / Find the output: `for (int i = 5; i >= 1; i--) { for (int j = 1; j <= i; j++) printf("%3d", j); }`

✎ **Asked in:** PGCB AE(CSE) 2021 · Cadet College Lecturer(ICT) 2025

```
for (int i = 5; i >= 1; i--) {
    for (int j = 1; j <= i; j++) cout << j << " ";
    cout << "\n";
}
```

1 2 3 4 5 · 1 2 3 4 · 1 2 3 · 1 2 · 1 (one row per line)

Exam trap: The Cadet College code has **no newline**: it prints all 15 numbers on one line, each 3 wide: ␣1␣2␣3␣4␣5␣1␣2␣3␣1.

```
for (int i = 5; i >= 1; i--) { for (int j = 1; j <= i; j++) printf("%3d", j); }
```

→ " 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1" (one line)

Q34: Read an integer and display a pattern: for 1234 print 1234, 234, 34, 4.

✎ **Asked in:** Hydrocarbon Unit AD(ICT) 2025

```
int n, p = 1;
scanf("%d", &n);
while (p <= n) p *= 10;            /* 10000 for 1234 */
for (p /= 10; p >= 1; p /= 10)
    printf("%d\n", n % (p * 10)); /* keep the last digits */
```

input 1234 → 1234 · 234 · 34 · 4

Q35: Print Floyd's (binary) triangle for N: n = 5 → 1 / 01 / 101 / 0101 / 10101.

✎ **Asked in:** Combined Bank AP 2024

```
int n;
scanf("%d", &n);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) printf("%d", (i + j) % 2 == 0);
    printf("\n");
}
```

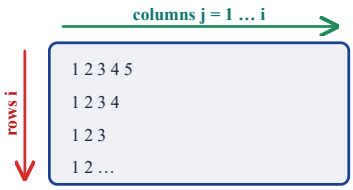
Cell (i, j) is 1 when i + j is even. → 1 · 01 · 101 · 0101 · 10101

Q36: Write a program (or pseudocode) that outputs "EXAM" 100 times.

✎ **Asked in:** Parliament Secretariat Programmer 2023

`for (int i = 1; i <= 100; i++) printf("EXAM\n");` — pseudocode: *FOR i ← 1 TO 100: PRINT "EXAM".*

Memory trick: Any pattern = **outer loop for rows, inner loop for columns**. Ask two questions: how many rows (i), and what does row i print (j from … to …)? Put `\n` after the inner loop.



14 Sum, product, odd, even, squares

Q37: Write a C program to find the sum of 1 to N. / Write C programs for the product and the sum of the numbers 1 to 100.
✎ **Asked in:** 15th NTRCA Computer Operation (2019) · WZPDCL SAE 2023

```c
int n;  long long sum = 0;  double prod = 1;
scanf("%d", &n);
for (int i = 1; i <= n; i++) { sum += i; prod *= i; }
printf("sum = %lld, product = %g\n", sum, prod);
```

n = 100 → sum = 5050, product = 9.33262e+157

Exam trap: 100! has 158 digits: `int` and `long long` overflow (13! already beats `int`). Use **double** and say so. Check the sum with n(n+1)/2.

Q38: What is a keyword? Write a C program for the sum of all odd numbers 1 + 3 + 5 + … up to 100. / Sum of the even numbers from 1 to 100. / Sum of all even numbers from 1 to n (C or Python).
✎ **Asked in:** 17th NTRCA CSE (2023) · BCPCL JI 2019 · Combined Bank Officer(IT) 2025

```c
int odd = 0, even = 0;
for (int i = 1; i <= 100; i += 2) odd += i;     /* 1,3,…,99 */
for (int i = 2; i <= 100; i += 2) even += i;    /* 2,4,…,100 */
printf("odd = %d, even = %d\n", odd, even);
```

```python
n = int(input())
print(sum(range(2, n + 1, 2)))
```

odd = 2500, even = 2550 · Python n = 10 → 30

A **keyword** is a reserved word with a fixed meaning that cannot be an identifier: `int, if, else, for, while, return, void, struct` (C has 32 in C89).

Q39: Write a program for $1^2 + 2^2 + 3^2 + \dots + N^2$ using a user-defined function. / Write a C++ program for 1 + (1+2) + (1+2+3) + … + (1+2+…+n).
✎ **Asked in:** 15th NTRCA CSE · MM AP 2017

```c
/* long sumsq(int n): loop s += i*i, return s */
int n;  scanf("%d", &n);
printf("%ld\n", sumsq(n));
```

```cpp
int n;  long term = 0, sum = 0;
cin >> n;
for (int i = 1; i <= n; i++) { term += i; sum += term; }
cout << sum << "\n";
```

N = 5 → 55 (= n(n+1)(2n+1)/6) · n = 4 → 1 + 3 + 6 + 10 = 20

Q40: Find the sum of the series 1 + 2 + 4 + 7 + 11 + … + N (to the n-th term).
✎ **Asked in:** Combined Bank AP 2024

```c
int n, term = 1, sum = 0;
scanf("%d", &n);
for (int i = 1; i <= n; i++) { sum += term; term += i; }  /* gaps 1,2,3,… */
printf("%d\n", sum);
```

Each term adds one more than the last gap. **Assumption**: N = number of terms (the second copy says "n-th term"). n = 5 → 1+2+4+7+11 = 25

Memory trick: Every series program is the same **three lines**: sum = 0 · loop over the terms · sum += term. Only "term" changes.

15 e^x, sin x, harmonic and fraction series

Q41: Write a program for the series $e^x = 1 + x/1! + x^2/2! + x^3/3! + \dots$
✎ **Asked in:** Sonali Bank ADBA 2024 · Combined Bank ADA 2024

```c
double x, term = 1, sum = 1;  int n;
scanf("%lf %d", &x, &n);              /* x and number of terms */
for (int i = 1; i < n; i++) {
    term = term * x / i;              /* x^i / i! from the last term */
    sum += term;
}
printf("e^%.1f = %.6f\n", x, sum);
```

x = 1, 15 terms → e^1.0 = 2.718282

Build each term from the **previous term** (× x / i): no `pow()`, no factorial overflow.

Q42: Write C code for $x - x^3/3! + x^5/5! - x^7/7! + \ldots$

✎ **Asked in:** PGCB AE(CSE) 2024

```
double x, term, sum;  int n;
scanf("%lf %d", &x, &n);
term = sum = x;
for (int i = 1; i < n; i++) {
    term = -term * x * x / ((2 * i) * (2 * i + 1));
    sum += term;
}
printf("sin(%.1f) = %.6f\n", x, sum);
```

This is the Taylor series of **sin x** (x in radians); each term = previous × $(-x^2)$ / ((2i)(2i+1)). x = 1 → sin(1.0) = 0.841471

Q43: Write a program for 1 + 1/2 + 1/3 + … + 1/n, n read from input. / What is an array processor? Write a C program for 1 + 1/2 + 1/3 + … + 1/N.

✎ **Asked in:** Fisheries & Livestock Instructor(CS) 2023 · 16th NTRCA CSE (2019)

```
int n;  double s = 0;
scanf("%d", &n);
for (int i = 1; i <= n; i++) s += 1.0 / i;
printf("%.6f\n", s);
```

Write `1.0 / i`: `1 / i` is integer division and gives 0 after the first term. n = 5 → 2.283333 An **array processor** (vector/SIMD processor) applies one instruction to many data elements in parallel.

Q44: Write a C/C++ program for the series up to n: 1/(2×3) + 2/(3×4) + 3/(4×5) + …

✎ **Asked in:** NESCO AM(ICT) 2021

```
int n;  double s = 0;
scanf("%d", &n);
for (int i = 1; i <= n; i++) s += (double) i / ((i + 1) * (i + 2));
printf("%.4f\n", s);
```

n = 3 → 1/6 + 2/12 + 3/20 = 0.4833

Q45: Write a program to find the minimum number from a series.

✎ **Asked in:** BTCL AE(CSE) 2017

```
int n, x, min;
scanf("%d %d", &n, &min);          /* n, then the first number */
for (int i = 1; i < n; i++) { scanf("%d", &x); if (x < min) min = x; }
printf("Minimum = %d\n", min);
```

Start `min` at the **first** number, not 0 (0 would be wrong for all-positive input). 5 numbers 7 −2 9 0 4 → Minimum = -2

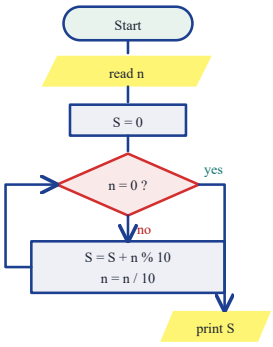
16 Working digit by digit: n % 10 and n / 10

Q46: Write a C program to sum all the digits of a 5-digit number. Draw the flowchart of this program. / Write a program in C for the sum of the digits of a number. 4 / Write the function `sumdigits` for the given `main` (input 123 → output 6).

✎ **Asked in:** 45th BCS CSE (2024) · Parliament Secretariat SAE 2023 · BTCL AM(Technical) 2021

```
int sumdigits(int n);              /* defined below */
int n;
scanf("%d", &n);
if (n < 10000 || n > 99999) printf("Not a 5-digit number\n");
else printf("Sum of digits of %d = %d\n", n, sumdigits(n));
/* int sumdigits(int n) { int s = 0;
       while (n != 0) { s += n % 10; n /= 10; }   // last digit, drop it
       return s; }                                                  */
```

input 12345 → Sum of digits of 12345 = 15



Q47: Find the sum of the digits of a number until the sum is a single digit (12345 → 6). / Find the summation of a number until it becomes a single digit. / Count the number of digits in a given number.

✎ **Asked in:** Combined Bank SO 2020 · Sonali Bank Officer(IT) 2021 · BKSP AP 2022

```
int n, s, digits = 0, t;
scanf("%d", &n);
for (t = n; t != 0; t /= 10) digits++;      /* count digits */
while (n > 9) {                            /* digital root */
```

12345 → 15 → 6. Short cut: digital root = 1 + (n − 1) % 9 for n > 0. → digits = 5, single digit = 6 (input 0 has 1 digit: handle it separately).

```
    for (s = 0; n != 0; n /= 10) s += n % 10;
    n = s;
}
printf("digits = %d, single digit = %d\n", digits, n);
```

Q48: A string contains letters and digits; print the sum of all the digit characters. / Score each letter (A 1, B 2, C 3, D 4, E 5, F 1, G 2, … Z 1) and sum the scores of a word; non-letters score nothing.

✎ **Asked in:** Jamuna Oil JO(IT & MIS) 2025 · DG Food AP 2022

```
char s[100], w[100];  int digits = 0, score = 0;
scanf("%s %s", s, w);
for (int i = 0; s[i]; i++)
    if (isdigit(s[i])) digits += s[i] - '0';      /* '7' - '0' = 7 */
for (int i = 0; w[i]; i++)
    if (w[i] >= 'A' && w[i] <= 'Z') score += (w[i] - 'A') % 5 + 1;
printf("digit sum = %d, score = %d\n", digits, score);
```

Character arithmetic: `c - '0'` turns a digit character into its value; the score repeats every 5 letters, so (position % 5) + 1. a1b2c3x9 → 15 · HELLO → 3+5+2+2+5 = 17

17 Reverse a number, palindrome number

Q49: Write a function in a structured language to reverse an integer, with the prototype `int reverse_int(int num)` (4567 → 7654). / Write a C program to reverse an integer. / Find the reverse for one positive and one negative input.

✎ **Asked in:** 46th BCS ICT · BCC AP 2017 · BAERA Engineer(CSE) January 2021

```
/* int reverse_int(int num) {
       int rev = 0;
       while (num != 0) { rev = rev * 10 + num % 10; num /= 10; }
       return rev; }                                          */
printf("%d %d %d\n", reverse_int(4567), reverse_int(-123), reverse_int(220));
```

→ 7654 -321 22

In C, `-123 % 10` = −3 and `-123 / 10` = −12 (truncation toward zero), so a negative number reverses with its sign: **−321**. (The prototype's `reverse-int` is not a legal name; use `reverse_int`.)

Q50: Find the output for input 6789 (and for any input): `while (number != 0) { remainder = number % 10; rev = rev * 10 + remainder; number /= 10; } printf("%d", rev);` / the same as a function `reverseNumber(num)`.

✎ **Asked in:** BGDCL AM(CSE) 2024

pass	number	remainder	rev = rev × 10 + rem
1	6789	9	9
2	678	8	98
3	67	7	987
4	6	6	**9876**

→ Reversed Number: 9876 · the function version: Input: 6789 / Output: 9876

Q51: Write a C/C++/Java program to reverse the digits of a number. Sample input 220, sample output 022.

✎ **Asked in:** JBL ANE 2022

```
int n;
scanf("%d", &n);
do { printf("%d", n % 10); n /= 10; } while (n != 0);
printf("\n");
```

The sample wants **022**: an `int` cannot hold a leading 0, so **print each digit** instead of building a number. 220 → 022

Q52: Check whether a given integer is a palindrome or not.

✎ **Asked in:** BUET AP(Exam Controller) 2025

```
int n, t, rev = 0;
scanf("%d", &n);
for (t = n; t > 0; t /= 10) rev = rev * 10 + t % 10;
printf("%d is %sa palindrome\n", n, (n == rev) ? "" : "not ");
```

A palindrome reads the same reversed: compare n with its reverse (keep the original in n, reverse a copy t). 12321 → is a palindrome; 1231 → not.

18 Prime, Armstrong, binary, case, quadratic

Q53: Write an algorithm / a C program to check whether a number is prime. / Show the prime numbers from 1 to n (n from user input; sample n = 20 → Prime Numbers: 2, 3, 5, 7, 11, 13, 17, 19,).

✎ **Asked in:** NSDA AP 2022 · Parliament Secretariat SAE 2023 · ICB AMCL AP 2024 · Combined Bank AP 2024 · Combined Bank SO(IT) 2024

```
/* int isPrime(int n) { if (n < 2) return 0;
       for (int i = 2; i * i <= n; i++) if (n % i == 0) return 0;
       return 1; }                                              */
int n;
printf("Enter value of n: ");  scanf("%d", &n);
printf("Prime Numbers: ");
for (int i = 2; i <= n; i++) if (isPrime(i)) printf("%d, ", i);
```

→ Enter value of n: Prime Numbers: 2, 3, 5, 7, 11, 13, 17, 19,

Algorithm: read n; if n < 2 → not prime; for i = 2 while i × i ≤ n: if n mod i = 0 → not prime, stop; otherwise prime. Testing up to √**n** is enough (the printed NSDA answer goes to n/2: correct but slower).

Q54: Write a program to check whether a number is Armstrong or not. / Binary to decimal conversion program (C/C++). / Write a C program for upper-case to lower-case conversion.

✎ **Asked in:** Combined Bank SO(IT) 2020 · DUET AP 2025 · DMTCL AE(ICT) 2023

```
int n, t, d = 0, sum = 0, dec = 0;  char bin[40], s[50];
scanf("%d %s %s", &n, bin, s);
for (t = n; t; t /= 10) d++;                              /* number of digits */
for (t = n; t; t /= 10) sum += (int) pow(t % 10, d);       /* 1^3 + 5^3 + 3^3 */
printf("%d is %sArmstrong\n", n, sum == n ? "" : "not ");
for (int i = 0; bin[i]; i++) dec = dec * 2 + (bin[i] - '0');   /* 1011 → 11 */
printf("binary %s = %d\n", bin, dec);
for (int i = 0; s[i]; i++) if (s[i] >= 'A' && s[i] <= 'Z') s[i] += 32;   /* 'A' 65 → 'a' 97 */
printf("%s\n", s);
```

→ 153 is Armstrong · binary 1011 = 11 · hello

Armstrong number: sum of each digit raised to the number of digits equals the number (153 = 1 + 125 + 27; also 370, 371, 407). Binary → decimal: **double and add** each bit (Horner). Upper → lower: add 32, or `tolower()`.

number	test in one line	examples
prime	no divisor from 2 to √n	2, 3, 5, 7, 11
Armstrong	Σ digit^k = n (k = number of digits)	153, 370, 371, 407
perfect	sum of proper divisors = n	6, 28, 496
palindrome	n = reverse(n)	121, 12321
strong	Σ digit! = n	145 = 1! + 4! + 5!

19 $ax^2 + bx + c = 0$: flowchart and program

Q55: Draw the flowchart and write C code to find the roots (value of x) of $ax^2 + bx + c = 0$.

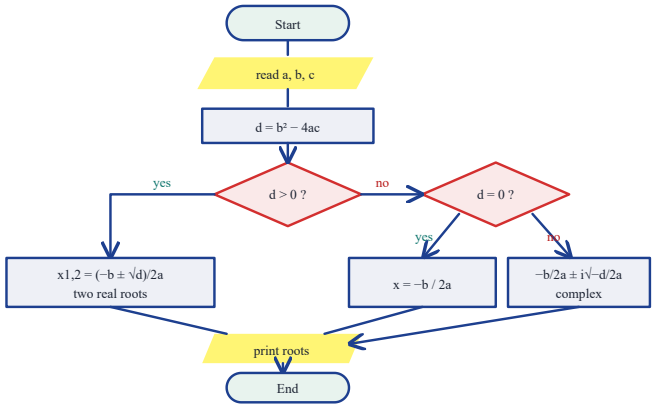
✎ **Asked in:** 16th NTRCA · 16th NTRCA ICT

```
double a, b, c, d;
scanf("%lf %lf %lf", &a, &b, &c);
d = b * b - 4 * a * c;
if (d > 0)       printf("x1 = %.2f, x2 = %.2f\n", (-b + sqrt(d)) / (2 * a), (-b - sqrt(d)) / (2 * a));
else if (d == 0) printf("x = %.2f\n", -b / (2 * a));
else             printf("x = %.2f ± %.2fi\n", -b / (2 * a), sqrt(-d) / (2 * a));
```

1 −3 2 → x1 = 2.00, x2 = 1.00 · 1 2 1 → x = -1.00 · 1 2 5 → x = -1.00 ± 2.00i

The **discriminant** $d = b^2 - 4ac$ decides: d > 0 two real roots, d = 0 one, d < 0 complex. Check **a ≠ 0** first (else it is linear).

a, b, c	$d = b^2 - 4ac$	case	roots (program output)
1, −3, 2	9 − 8 = 1	d > 0	x1 = 2.00, x2 = 1.00
1, 2, 1	4 − 4 = 0	d = 0	x = −1.00 (one repeated root)
1, 2, 5	4 − 20 = −16	d < 0	x = −1.00 ± 2.00i



the flowchart NTRCA asks for, with the three cases of d

Memory trick: **% 10 peels, / 10 drops, × 10 + pushes** (digit programs) · **d decides** (quadratic): plus → 2, zero → 1, minus → complex.

20 Parameter passing: value vs reference

Q56: Differences between "call by value" and "call by reference". / What is parameter passing? Explain with examples. / Illustrate the difference in a function with examples.

✎ **Asked in:** 17th NTRCA ICT (2023) · BPSC ADA 2022 · BPSC AP 2022 · BPSC AP 2023 · PSB AP 2018 · 15th NTRCA · 15th NTRCA ICT · 13th NTRCA CSE (2016) · 46th BCS ICT

Point	Call by value	Call by reference
what is passed	a **copy of the value**	the **address** of the variable
change inside the function	does **not** affect the caller	**changes the caller's variable**
memory	a new copy	same location, no copy
C syntax	`void f(int x)` · `f(a)`	`void f(int *x)` · `f(&a)`
safety / speed	safe; slow for big data	fast; can change data by mistake

Q57: Write a simple program in C to swap two integers using call by value. / Mention two basic differences between call by value and call by reference; write a program to swap two integers using call by reference.

✎ **Asked in:** BPSC AP 2023 · 46th BCS CSE

```c
#include <stdio.h>
void swapv(int x, int y) { int t = x; x = y; y = t;
    printf("inside swapv: %d %d\n", x, y); }
void swapr(int *x, int *y) { int t = *x; *x = *y; *y = t; }
int main(void) {
    int a = 10, b = 20;
    swapv(a, b);  printf("after swapv: %d %d\n", a, b);
    swapr(&a, &b); printf("after swapr: %d %d\n", a, b);
    return 0;
}
```

→ inside swapv: 20 10
→ after swapv: 10 20
→ after swapr: 20 10

Exam trap: "Swap using call by value" can only swap the **copies**: print inside the function to show it worked there and that the caller is unchanged.

Q58: Write a C code segment / program to swap two numbers without using a third variable.

✎ **Asked in:** WZPGCL AE(CSE) 2023 · 45th BCS ICT (2024)

```c
int a = 5, b = 9;
a = a + b;   /* 14 */
b = a - b;   /* 5  */
a = a - b;   /* 9  */
printf("a=%d b=%d\n", a, b);
```

Or with XOR: `a ^= b; b ^= a; a ^= b;` (no overflow risk). → a=9 b=5

- **Call by name** (ALGOL): the argument *expression* is re-evaluated each time the parameter is used; call by value copies once; call by reference passes the address. Read the scenario: "change seen by the caller" → reference; "copy" → value; "expression evaluated again" → name. ✎ BAERA Engineer(CSE) November 2021
- `typedef struct { int roll; char name[28]; double cgpa; } S;` passed by value copies the **whole struct**, array included: 4 + 28 + 8 = **(b) 40** bytes ("arrays by reference" applies only to bare arrays). ✎ BUET M.Sc. April 2021

21 Functions: syntax, library vs user-defined

Q59: Write about the syntax of a function.
✎ **Asked in:** BARC Programmer 2023

```c
#include <stdio.h>
int add(int a, int b);            /* prototype (declaration) */
int main(void) {
    printf("%d\n", add(2, 3));     /* call */
    return 0;
}
int add(int a, int b) {           /* definition: header */
    return a + b;                 /* body */
}
```

`return_type name(parameter list) { body; return value; }`. Parts: **prototype** (tells the compiler the types), **definition**, **call**. A function without a result has return type `void`. → 5

Q60: Differences between user-defined and library functions in C.
✎ **Asked in:** 17th NTRCA · 17th NTRCA ICT (2023)

Library (built-in)	User-defined
already written, in the C library	written by the programmer
declared in a header: `#include <math.h>`	needs our own prototype and definition
name and job fixed	any name, any job
`printf()`, `scanf()`, `sqrt()`, `strlen()`	`add()`, `isPrime()`, `factorial()`

Q61: Write a function that checks whether a year is a leap year.
✎ **Asked in:** BUET M.Sc. May 2016

```c
/* int isLeap(int y) {
       return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; } */
printf("%d %d %d %d\n", isLeap(2024), isLeap(1900), isLeap(2000), isLeap(2023));
```

Divisible by 4, except centuries, unless divisible by 400: **1900 no, 2000 yes**. → 1 0 1 0

MCQ as asked	answer	asked in
`a = func3(6) - func2(4,5) / func1(1,2,3);` Which function is called first?	**(d) cannot be predicted**	BUET IICT April 2021
Python function (the stem was not preserved): `def add(a, b=0): return a + b` — `def`, default argument, `return`	—	Combined Bank Officer(IT) 2026

Precedence says *which operation* happens first, not *which operand is evaluated* first: C leaves the order of evaluating `func1`, `func2`, `func3` **unspecified**.

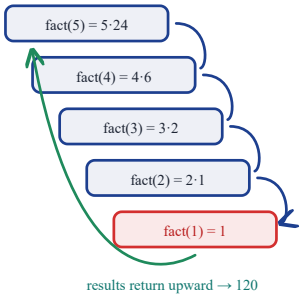
Memory trick: Value = **photocopy** (scribble on it, the original is safe). Reference = **the original file** (every change sticks).

22 Recursion and the recursive factorial

Q62: What do you mean by recursion? Calculate the factorial using recursion with C code. / Define recursion; write a recursive C factorial program. / Write a program for the factorial using the recursion method (C or Java). / Write a C program for the factorial of a positive integer.
✎ **Asked in:** 45th BCS ICT (2024) · BPSC ANE 2020 · BPSC AP 2020 · BPSC AP 2022 · Milk Vita AM(CSE) 2023 · BBA AME 2025 · ICT Ministry AP 2021 · 46th BCS ICT · 18th NTRCA College ICT (2024)

Recursion: a function that **calls itself** on a smaller version of the problem. Every recursive function needs a **base case** (stops the calls) and a **recursive case** that moves toward it; without a base case the stack overflows.

```c
#include <stdio.h>
long long fact(int n) {
    if (n <= 1) return 1;          /* base case */
    return n * fact(n - 1);        /* recursive case */
}
int main(void) {
    int n;
    scanf("%d", &n);
    printf("%d! = %lld\n", n, fact(n));
    return 0;
}
```



input 5 → 5! = 120

Q63: What is a recursive function? Give an example. / What are recursion and a recursive function? Explain with an example. / Explain recursion and parameter passing with examples.
✎ **Asked in:** WZPDCL AE(CSE) 2019 · 15th NTRCA · 15th NTRCA ICT · 17th NTRCA ICT (2023)

Use the factorial above as the example, and draw the call stack. Parameter passing in it is **by value**: each call gets its own copy of `n` on the stack (call by value vs reference: previous sheets).

Q64: How do recursion and iteration differ in stack usage and memory overhead? Explain with examples.
✎ **Asked in:** 47th BCS CSE

	Recursion	Iteration (loop)
stack	one **stack frame per call**: fact(n) holds n frames → O(n) memory	one frame, a few variables → O(1)
overhead	call/return cost, risk of **stack overflow** for deep n	none beyond the loop
code	short, natural for trees, divide-and-conquer	longer for such problems

23 Fibonacci, GCD, digit sum, palindrome

Q65: Print the Fibonacci series without recursion and with recursion. / Display the first ten Fibonacci numbers. / Find the n-th Fibonacci number using recursion. / Show the Fibonacci series from 0 up to n.
✎ **Asked in:** 17th NTRCA CSE (2023) · BB AP 2016 · 17th NTRCA ICT (2023) · 17th NTRCA · KGDCL AE(CSE) 2021 · NWPGCL JAM 2020

```c
#include <stdio.h>
int fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
int main(void) {
    int a = 0, b = 1, c;
    for (int i = 0; i < 10; i++) {             /* loop */
        printf("%d ", a);  c = a + b;  a = b;  b = c;
    }
    printf("\nfib(9) = %d\n", fib(9));          /* recursion */
    return 0;
}
```

→ 0 1 1 2 3 5 8 13 21 34 · fib(9) = 34 The recursive version is **exponential** (fib(n−1) and fib(n−2) recompute the same values); the loop is O(n). "0 up to n": loop while `a <= n`.

Q66: Write a recursive C function for the GCD of two integers; using it, explain the steps for GCD(96, 36).
✎ **Asked in:** 47th BCS CSE

```c
/* int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); } */
printf("%d\n", gcd(96, 36));
```

gcd(96, 36) → gcd(36, 24) → gcd(24, 12) → gcd(12, 0) → **12**. → 12

Q67: Write a recursive function that returns the sum of the digits of a number (3426 → 15). / Write a function that separates each digit of an integer; you must use recursion.
✎ **Asked in:** GTCL AE(CSE) 2022 · Parliament Secretariat SAE 2023 · MoFood Network/Website Manager 2025 · BUET M.Sc. April 2017

```c
/* int sumd(int n) { return n == 0 ? 0 : n % 10 + sumd(n / 10); }
   void digits(int n) { if (n >= 10) digits(n / 10);
                        printf("%d ", n % 10); }   // print after the call */
printf("%d\n", sumd(3426));  digits(3426);
```

Printing **after** the recursive call gives the digits left to right. → 15 · 3 4 2 6

Q68: Write a recursive function that takes a string and returns a Boolean: is it a palindrome? / Write a recursive function for the scenario `for (i = 1; i < n; i++) for (j = 0; j < i; j++) for (k = 0; k < i; k++) x = x + 1;`

✎ **Asked in:** Combined Bank SO(IT) 2022 · Titas Gas AE(CSE) 2024

```
/* int pal(const char *s, int i, int j) {
       if (i >= j) return 1;              // met in the middle
       if (s[i] != s[j]) return 0;
       return pal(s, i + 1, j - 1); }
   int count(int n) { return n <= 1 ? 0 : count(n - 1) + (n - 1) * (n - 1); } */
printf("%d %d %d\n", pal("madam", 0, 4), pal("hello", 0, 4), count(4));
```

For each i the two inner loops add i × i, so x = Σ i² for i = 1 … n−1: count(n) = count(n−1) + (n−1)². → 1 0 14 (n = 4: 1 + 4 + 9).

24 Arrays: declare, pass, size, min/max, reverse

Q69: Differentiate between the array and structure data types. / Write the difference between structure and array.

✎ **Asked in:** BOF AE(CSE) 2021 · BPSC AE 2022 · BPSC AP 2021 · BPSC AP 2022

Array	Structure
collection of elements of the **same type**	collection of members of **different types**
accessed by **index**: `a[2]`	accessed by **name**: `s.roll`
derived type, no keyword	user-defined, keyword `struct`
array name = address of element 0	structure name is not a pointer
`int marks[5];`	`struct student { int roll; char name[20]; float cgpa; };`

Q70: How can we initialise a 1-D array? How can we pass an array to a function in C?

✎ **Asked in:** 45th BCS CSE (2024)

```
#include <stdio.h>
int minOf(int a[], int n) {            /* also: int *a */
    int m = a[0];
    for (int i = 1; i < n; i++) if (a[i] < m) m = a[i];
    return m;
}
int main(void) {
    int a[] = {7, 3, 9, 1, 5};          /* size from the list */
    int b[5] = {1, 2};                  /* rest become 0 */
    printf("min = %d, b[4] = %d\n", minOf(a, 5), b[4]);
    return 0;
}
```

Initialise at declaration (`int a[5] = {…}`, size may be omitted, missing values become 0) or in a loop. An array is passed as the **address of its first element**, so pass the length too; the function can change the caller's elements. → min = 1, b[4] = 0

- Passing an array to a function passes: **C. the address of the first element**. ✎ BPSC AME 2019
- Size of `int array[15]` with 4-byte int: 15 × 4 = **60**. ✎ BPSC AP 2022
- Address difference between the 1st and 4th `float` (4 bytes): 3 × 4 = **(a) 12**. ✎ BUET IICT April 2021
- Python **list** holds any mix of types and grows freely; an **array** (`array` module / NumPy) holds one type, uses less memory and supports fast element-wise maths. ✎ WZPDCL SAE 2023

Exam trap: C never checks array bounds: for `int a[5]` the valid indexes are **0 … 4**; `a[5]` reads past the end (undefined behaviour), and a loop written `i <= n` instead of `i < n` does exactly that.

25 Array programs: min/max, reverse, averages

Q71: Write a function returning the minimum (smallest) value of an array. / Read 10 array elements and find the maximum. / Find the maximum price of 20 items. / Read N numbers and print the largest and the smallest.

✎ **Asked in:** BSCL AE(CSE) 2025 · Combined Bank AP 2023 · BBA AME 2025 · BB AP 2019 · BKSP AP 2022

```
int n, a[100], max, min;
scanf("%d", &n);
for (int i = 0; i < n; i++) scanf("%d", &a[i]);
max = min = a[0];
for (int i = 1; i < n; i++) {
    if (a[i] > max) max = a[i];
    if (a[i] < min) min = a[i];
```

Same loop for 10 elements or 20 prices: set n. 5 numbers 40 15 90 25 60 → max = 90, min = 15

```
}
printf("max = %d, min = %d\n", max, min);
```

Q72: Write a C program that stores some data in an array and prints them in reverse order.
✎ **Asked in:** 17th NTRCA ICT (2023) · 17th NTRCA

```
int n, a[100];
scanf("%d", &n);
for (int i = 0; i < n; i++) scanf("%d", &a[i]);
for (int i = n - 1; i >= 0; i--) printf("%d ", a[i]);
```

10 20 30 40 50 → 50 40 30 20 10

Q73: Read five numbers into an array and find their average. / Find the sum, product and average of numbers in an array. / N students' marks: find the average and list who is above and below it.
✎ **Asked in:** 16th NTRCA ICT · 16th NTRCA · BOF AP 2022

```
int n;  double m[100], sum = 0, prod = 1, avg;
scanf("%d", &n);
for (int i = 0; i < n; i++) { scanf("%lf", &m[i]); sum += m[i]; prod *= m[i]; }
avg = sum / n;
printf("sum %.0f, product %.0f, average %.2f\n", sum, prod, avg);
for (int i = 0; i < n; i++)
    printf("student %d: %s\n", i + 1, m[i] >= avg ? "above" : "below");
```

70 45 88 60 52 → sum 315, product 864864000, average 63.00 · 1 above, 2 below, 3 above, 4 below, 5 below

Q74: A function receives an integer array and prints the numbers divisible by 3. / Read 10 integers and find the average of those divisible by 5.
✎ **Asked in:** Combined Bank Officer(IT) 2024 · DESCO SAE 2022

```
/* void div3(int a[], int n) { for (…) if (a[i] % 3 == 0) printf("%d ", a[i]); } */
int a[10] = {3, 10, 12, 25, 7, 30, 18, 5, 40, 9}, cnt = 0, sum = 0;
div3(a, 10);
for (int i = 0; i < 10; i++) if (a[i] % 5 == 0) { sum += a[i]; cnt++; }
if (cnt) printf("average of multiples of 5 = %.2f\n", (double) sum / cnt);
```

→ 3 12 30 18 9 · average of multiples of 5 = 22.00 (10, 25, 30, 5, 40)

26 Missing number, common elements, Kadane

Q75: An array of size 10 holds 1 to 10 in random order, but one number was replaced by 0. Write a C/C++ program (with functions) to restore the lost number.
✎ **Asked in:** BAERA Engineer(CSE) 2023 · MoFood Network/Website Manager 2025

```
/* int restore(int a[], int n): s = sum of a; zero at index z;
      a[z] = n(n+1)/2 - s;  return a[z]; */
int a[10] = {4, 9, 1, 0, 7, 2, 10, 3, 8, 5};
printf("missing = %d\n", restore(a, 10));
```

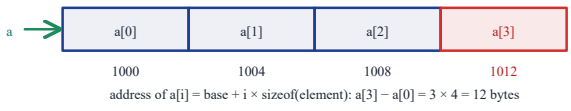
1 + … + 10 = **55**; the array sums to 49, so the zero was 6. One pass, O(n), no sorting. → missing = 6

Q76: Print the common elements of two arrays and their count (elements distinct). / Report whether an integer array is ascending, descending or unordered. / Maximum subarray: find the contiguous subarray with the largest sum in [−2, 1, −3, −1, 2, 1, −5, 4]. / Array {4,5,6,7}: print every pair {4,5}, {4,6},…,{6,7}.
✎ **Asked in:** BB AD(ICT) 2025 · BUET M.Sc. · BPDB AE(CSE) 2023 · NWPGCL AE(CSE)

```
int A[] = {1, 4, 6, 9}, B[] = {2, 4, 9, 11}, c = 0;
for (int i = 0; i < 4; i++) for (int j = 0; j < 4; j++)
    if (A[i] == B[j]) { printf("%d ", A[i]); c++; }
printf("(count %d)\n", c);
int s[] = {2, 1, 0, -1, -2, -3}, up = 1, down = 1;
for (int i = 1; i < 6; i++) { if (s[i] <= s[i-1]) up = 0; if (s[i] >= s[i-1]) down = 0; }
printf("%s\n", up ? "ascending" : down ? "descending" : "unordered");
int m[] = {-2, 1, -3, -1, 2, 1, -5, 4}, best = m[0], cur = m[0];
for (int i = 1; i < 8; i++) {                       /* Kadane */
    cur = (cur + m[i] > m[i]) ? cur + m[i] : m[i];
    if (cur > best) best = cur;
}
printf("max subarray sum = %d\n", best);
int p[] = {4, 5, 6, 7};
for (int i = 0; i < 4; i++) for (int j = i + 1; j < 4; j++) printf("{%d,%d} ", p[i], p[j]);
```

→ 4 9 (count 2) · descending · max subarray sum = 4 · {4,5} {4,6} {4,7} {5,6} {5,7} {6,7}

Kadane's algorithm (O(n)): keep the best sum ending here (extend or restart) and the best seen. Here the answer is the single element **[4]**; [2, 1] gives only 3.



an array is one block; its name is the address of a[0]

27 Matrix programs, sorting and searching

Q77: Take matrices A and B as input (handle different dimensions): (a) C = A × B (b) the average of A and of B (c) the maximum of C.
✎ **Asked in:** Combined Bank SO(IT) 2023

```
int r1, c1, r2, c2;  double A[10][10], B[10][10], C[10][10], sa = 0, sb = 0, mx;
scanf("%d %d", &r1, &c1);
for (int i = 0; i < r1; i++) for (int j = 0; j < c1; j++) { scanf("%lf", &A[i][j]); sa += A[i][j]; }
scanf("%d %d", &r2, &c2);
for (int i = 0; i < r2; i++) for (int j = 0; j < c2; j++) { scanf("%lf", &B[i][j]); sb += B[i][j]; }
if (c1 != r2) { printf("Cannot multiply: columns of A != rows of B\n"); return 0; }
for (int i = 0; i < r1; i++) for (int j = 0; j < c2; j++) {
    C[i][j] = 0;
    for (int k = 0; k < c1; k++) C[i][j] += A[i][k] * B[k][j];
}
mx = C[0][0];
for (int i = 0; i < r1; i++) { for (int j = 0; j < c2; j++) { printf("%6.0f", C[i][j]); if (C[i][j] > mx) mx = C[i][j]; } printf("\n"); }
printf("avg A %.2f, avg B %.2f, max C %.0f\n", sa / (r1 * c1), sb / (r2 * c2), mx);
```

A (2×3) = 1 2 3 / 4 5 6, B (3×2) = 7 8 / 9 10 / 11 12 → C = 58 64 / 139 154 · avg A 3.50, avg B 9.50, max C 154

Q78: Read an m × n matrix (m, n from the user) and print each row followed by its sum, then the column sums (sample on the right).
✎ **Asked in:** Combined Bank SO(IT) 2024

```
int m, n, a[10][10], col[10] = {0};
scanf("%d %d", &m, &n);
for (int i = 0; i < m; i++) {
    int row = 0;
    for (int j = 0; j < n; j++) { scanf("%d", &a[i][j]); row += a[i][j]; col[j] += a[i][j]; printf("%d ", a[i][j]); }
    printf("%d\n", row);
}
for (int j = 0; j < n; j++) printf("%d ", col[j]);
```

→ 1 3 4 2 10
→ 2 4 5 3 14
→ 3 2 2 1 8
→ 6 9 11 6 (column sums)

Q79: Write a C program to sort in ascending order using selection sort. / Write a C/C++ program for binary search.
✎ **Asked in:** BCC AP 2017 · Petrobangla AM(IT) 2022

```
int a[] = {64, 25, 12, 22, 11}, n = 5, key = 22, lo = 0, hi = 4, mid, t;
for (int i = 0; i < n - 1; i++) {                /* selection sort */
    int m = i;
    for (int j = i + 1; j < n; j++) if (a[j] < a[m]) m = j;
    t = a[i]; a[i] = a[m]; a[m] = t;             /* smallest to front */
}
for (int i = 0; i < n; i++) printf("%d ", a[i]);
while (lo <= hi) {                               /* binary search */
    mid = (lo + hi) / 2;
    if (a[mid] == key) { printf("\nfound %d at index %d\n", key, mid); break; }
    if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
}
```

→ 11 12 22 25 64
→ found 22 at index 2

Selection sort: n − 1 passes, smallest to the front: $O(n^2)$. Binary search needs a **sorted** array, halves the range: O(log n).

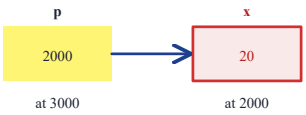
28 Pointers, malloc vs calloc, DMA

Q80: What is a pointer? Write the advantages (and disadvantages) of using pointers. / Why are pointers needed? How is a pointer declared?
✎ **Asked in:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 17th NTRCA · 15th NTRCA CSE · 15th NTRCA ICT

Pointer: a variable that stores the **memory address** of another variable. Declare `type *name;` · `&x` = address of x · `*p` = value at the address (dereference).

```
int x = 10;
int *p = &x;          /* p holds x's address */
*p = 20;              /* changes x through p */
printf("x = %d, *p = %d\n", x, *p);
```

→ x = 20, *p = 20



p stores the address of x

Advantages

- ✓ call by reference: a function can change the caller's data
- ✓ **dynamic memory** (malloc) and data structures (linked list, tree)
- ✓ fast array and string handling
- ✓ return several values; pass big structs cheaply

Disadvantages

- ✓ uninitialised / dangling pointers crash the program
- ✓ **memory leaks** if `free` is forgotten
- ✓ hard to read and debug
- ✓ can overwrite any memory

Q81: What is a pointer? Write the differences between malloc() and calloc(). / Describe dynamic memory allocation in C. / How does dynamic memory allocation help in managing data?

✎ **Asked in:** 15th NTRCA ICT · 17th NTRCA ICT (2023) · 15th NTRCA · SPCBL SAE 2022 · 37th BCS

	`malloc(size)`	`calloc(n, size)`
arguments	one: total bytes	two: count and size of each
initial value	**garbage**	all bytes **0**
speed	faster	a little slower (zeroing)

```
int n = 5, *a = calloc(n, sizeof(int));   /* zeroed */
if (a == NULL) return 1;                  /* check! */
for (int i = 0; i < n; i++) a[i] = i * i;
a = realloc(a, 2 * n * sizeof(int));      /* grow */
printf("%d %d\n", a[4], n * 2);
free(a);                                  /* give back */
```

Dynamic memory allocation reserves memory on the **heap at run time** (`malloc, calloc, realloc`) and returns it with `free`, so a program uses exactly as much as the data needs (unknown n, growing lists) instead of a fixed array. → 16 10

29 Pointer arithmetic and reading declarations

Q82: How does pointer arithmetic behave in C? Illustrate with an example. / Five short questions on pointer arithmetic, including multidimensional arrays.

✎ **Asked in:** 47th BCS CSE · BUET M.Sc. October 2017

```
int a[5] = {10, 20, 30, 40, 50}, *p = a;
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d %d %d\n", *(p + 2), p[4], *p + 1);     /* 30 50 11 */
printf("%ld\n", (long) (&a[4] - &a[0]));       /* elements apart: 4 */
printf("%d %d\n", *(*(m + 1) + 2), *(m[0] + 4));  /* m[1][2], m[1][1] */
```

`p + 1` moves by **sizeof(*p)** bytes, not 1 byte; subtracting two pointers gives the number of **elements** between them. `*(*(m + i) + j)` = `m[i][j]`; rows are stored one after another, so `m[0] + 4` lands on m[1][1]. → 30 50 11 · 4 · 6 5

MCQ as asked	answer	asked in
Declare an array Arr of four pointers to float	**`float *Arr[4];`**	49th BCS ICT (2025)
Function pointer f to a function taking (int, float) and returning double	**`double (*f)(int, float)`**	49th BCS ICT (2025)
`int (*a)[10];` declares …	**a pointer to an array of 10 ints**	BPSC AME 2019
Pass a function pointer: (a) `void pass(int (*fptr)(int, float, char)){}` (b) `void pass(*fptr(…)){}` …	**(a)**	BUET IICT April 2021
p → float array at 0x002a32e4; value of `&p[5]`?	**(c) 0x002a32f8**	BUET M.Sc. April 2021
Pair NOT treated as identical: `foo(int *i)`/`foo(int i[])` · `a[i]`/`i[a]` · `char c[10]`/`char *c` · `bar(int)`/`bar(int x)`	**`char c[10]; char *c;`**	BPSC AME 2022

```
float f[6];  float *p = f;
printf("%ld bytes\n", (long) ((char *) &p[5] - (char *) p));   /* 5 * 4 = 20 = 0x14 */
int a[3] = {7, 8, 9};  printf("%d %d\n", a[2], 2[a]);           /* a[i] == *(a+i) == i[a] */
```

→ 20 bytes (0x002a32e4 + 0x14 = 0x002a32f8) · 9 9

Memory trick: Read a declaration **from the name outward, right before left**: `int (*a)[10]` → a is a *pointer* to an *array of 10* ints; `float *Arr[4]` → Arr is an *array of 4 pointers* to float.

Exam trap: `a[i]` = `i[a]` = `*(a + i)`; a parameter `int i[]` is `int *i`. Only a **declared array** differs from a pointer.

30 Strings without library functions

Q83: Write a program to reverse a string without using a library function. / Write a function to reverse a string without a global variable. / Write a C++ method to reverse a char array without a temp array or STL functions. / Find the length of a string without `strlen()`.
✎ **Asked in:** BREB AP 2023 · Combined Bank ADA 2022 · BUET M.Sc. May 2019 · BUET M.Sc. May 2016 · BPSC AME 2021

```
/* int len(const char *s) { int n = 0; while (s[n] != '\0') n++; return n; }
   void reverse(char *s) {                  // in place: two ends meet
       for (int i = 0, j = len(s) - 1; i < j; i++, j--)
           { char t = s[i]; s[i] = s[j]; s[j] = t; } }        */
char s[] = "Bangladesh";
printf("%d ", len(s));  reverse(s);  printf("%s\n", s);
```

Length: count characters up to the terminating **'\0'**. Reverse: swap the two ends and move inward — in place, so no second array, no global (only one `char` for the swap). → 10 hsedalgnaB

Q84: Write a Java program counting the vowels and consonants in a given string.
✎ **Asked in:** BOF AP 2022

```
String s = "Programming in Java".toLowerCase();
int v = 0, c = 0;
for (char ch : s.toCharArray())
    if (ch >= 'a' && ch <= 'z') { if ("aeiou".indexOf(ch) >= 0) v++; else c++; }
System.out.println("vowels " + v + ", consonants " + c);
```

Count only letters: spaces and digits are neither. → vowels 6, consonants 11

Q85: Write a C or Java function converting a string to an integer without built-in conversion functions. / Replace each letter with the next letter (case preserved, z → a, non-letters unchanged) and state the time complexity.
✎ **Asked in:** DPDC AM(ICT) 2025 · ICB Securities AP 2026

```
/* int toInt(const char *s): optional '-', then v = v*10 + (c - '0')
   void shift(char *s): letter → (c - 'a' + 1) % 26 + 'a', same for A-Z */
char w[] = "Zebra-9y";
printf("%d %d\n", toInt("-4512") + 2, toInt("307"));
shift(w);  printf("%s\n", w);
```

Both are one pass over the string: **O(n)** time, O(1) extra space. → -4510 307 · Afcsb-9z

Q86: Write a C program that takes integers as command-line arguments and prints their sum.
✎ **Asked in:** DPDC AM(ICT) 2025

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {          /* argv[0] = program name */
    long sum = 0;
    for (int i = 1; i < argc; i++) sum += atol(argv[i]);
    printf("sum = %ld\n", sum);
    return 0;
}
```

Run `./sum 10 20 -5 7`: `argc` = 5, the numbers arrive as strings, converted with `atol`. → sum = 32

- **Array vs pointer**: an array is a fixed block (`sizeof a` = whole array, its name cannot be reassigned); a pointer is one variable holding an address (`sizeof p` = 8, can point anywhere, can be changed: `p++`). ✎ BPSC AME 2021

31 Remove a substring, IP class, IP validation

Q87: Write a C program to remove a given substring from a string, e.g. input "programming", remove "gram" → output "proming".
✎ **Asked in:** RPGCL AM(ICT) 2022

```
char s[100] = "programming", sub[] = "gram";
char *p;
int k = strlen(sub);
while ((p = strstr(s, sub)) != NULL)          /* find it */
```

→ proming

```
    memmove(p, p + k, strlen(p + k) + 1);     /* close the gap */
printf("%s\n", s);
```

Exam trap: The printed solution deletes every copy of one **character**, not the substring, so it cannot give "proming"; it also uses `conio.h` and `void main`.

Q88: Given an IPv4 address string, write C/C++/Java code to show the class the address belongs to (sample 192.168.0.1). / Identify the class of an IP address.

✎ **Asked in:** Rupali Bank ANE 2021 · JBL ANA 2022

```
/* char ipclass(const char *ip) {
       int a = atoi(ip);                // first octet
       return a < 128 ? 'A' : a < 192 ? 'B' : a < 224 ? 'C' : a < 240 ? 'D' : 'E'; } */
printf("%c %c %c\n", ipclass("192.168.0.1"), ipclass("10.0.0.5"), ipclass("172.16.4.2"));
```

class	first octet
A	0–127
B	128–191
C	192–223
D	224–239 (multicast)
E	240–255

→ C A B

Q89: Write a program that validates IPv4 addresses and prints "valid" or "not valid" (e.g. 128.0.0.1 and 125.16.100.1 valid; 125.512.100.1 and 125.512.100.abc not valid).

✎ **Asked in:** RPGCL AM(ICT) 2022

```
/* int valid(const char *ip): for each part: 1–3 digits, no leading 0, value ≤ 255;
      parts separated by '.', exactly 4 parts, nothing else */
const char *t[] = {"128.0.0.1", "125.16.100.1", "125.512.100.1", "125.512.100.abc", "1.2.3"};
for (int i = 0; i < 5; i++) printf("%s: %s\n", t[i], valid(t[i]) ? "valid" : "not valid");
```

→ 128.0.0.1 valid · 125.16.100.1 valid · 125.512.100.1 not valid · 125.512.100.abc not valid · 1.2.3 not valid

Rules to state: **four** parts separated by dots; each part only digits, 0–255, no leading zero ("01" is invalid); no empty part. Parse by hand (or `strtok` on ".") and check every rule.

32 Structure vs union, nested structure, structure + malloc

Q90: What are structure and union? Write their differences with examples. / Explain the differences between structure and union with examples. / Describe structure and union and how to declare them. / Write the rules for declaring a union.

✎ **Asked in:** 16th NTRCA · 16th NTRCA ICT · 47th BCS CSE · 15th NTRCA CSE · 13th NTRCA CSE (2016)

	Structure (`struct`)	Union (`union`)
memory	each member has **its own space**	all members **share one space**
size	sum of members (+ padding)	size of the **largest member**
values at once	all members	only the last one written
use	a record (student, employee)	save memory; one of several types

```
#include <stdio.h>
struct S { int i; char c; float f; };    /* struct tag { members }; */
union  U { int i; char c; float f; };    /* same syntax, keyword union */
int main(void) {
    union U u;
    u.i = 65;                           /* write int, read char */
    printf("%zu %zu %c\n", sizeof(struct S), sizeof(union U), u.c);
    return 0;
}
```

Declaring a union: `union tag { type m1; type m2; … };` then `union tag var;`, members reached with `.` (or `->` through a pointer); only one member is valid at a time. → 12 4 A (4 + 1 + 3 padding + 4).

Q91: What is a nested structure in C? Explain with an example.

✎ **Asked in:** SPCBL SAE 2022

```
struct date { int d, m, y; };
struct emp { char name[20]; struct date join; };   /* struct inside struct */
struct emp e = {"Rina", {1, 7, 2024}};
printf("%s joined %d/%d/%d\n", e.name, e.join.d, e.join.m, e.join.y);
```

A **nested structure** has a structure as a member; reach inner members with two dots. → Rina joined 1/7/2024

Q92: Write a program to store information using structures with dynamic memory allocation.
✎ **Asked in:** 38th BCS (2018)

```
struct course { int marks; char subject[30]; };
int n;
scanf("%d", &n);
struct course *p = malloc(n * sizeof(struct course));
if (p == NULL) return 1;
for (int i = 0; i < n; i++) scanf("%d %29s", &p[i].marks, p[i].subject);
for (int i = 0; i < n; i++) printf("%s: %d\n", (p + i)->subject, (p + i)->marks);
free(p);
```

input 2 · 75 Physics · 82 Chemistry →
Physics: 75
Chemistry: 82

33 getc vs getch, formatted I/O, files, argv

Q93: Differences between getc(), getchar(), getch() and getche(). / Difference between getch() and getche().
✎ **Asked in:** 45th BCS ICT (2024) · BBL AM(ICT) 2019

function	reads from	echo on screen?	waits for Enter?	header
`getc(fp)`	any stream / file	—	buffered	stdio.h
`getchar()`	stdin (= getc(stdin))	yes	**yes**	stdio.h
`getch()`	keyboard	**no**	no	conio.h (non-standard)
`getche()`	keyboard	**yes** (e = echo)	no	conio.h

- **File pointer**: `FILE *fp = fopen("data.txt", "r");` — a pointer to a `FILE` structure that keeps the file's position and buffer; used by `fscanf, fprintf, fgets, fclose`. ✎ BBL AM(ICT) 2019
- **Formatted I/O** = `printf` / `scanf` with format specifiers that convert between text and values. Keyboard input: `scanf("%d", &x);`. `%d` means a signed decimal `int` (others: `%f %c %s %lf`). ✎ BPSC ANE 2020

Q94: myfile.txt has lines "day, demand, shortage" (1, 43571, 443 / 2, 34554, 432 / …). Write a C program to find the power shortage of the 30th day.
✎ **Asked in:** BPDB AE(CSE) 2018

```
FILE *fp = fopen("myfile.txt", "r");
int day, demand, shortage;
if (fp == NULL) { printf("cannot open\n"); return 1; }
while (fscanf(fp, "%d, %d, %d", &day, &demand, &shortage) == 3)
    if (day == 30) { printf("Day 30 shortage = %d\n", shortage); break; }
fclose(fp);
```

sample file with day 30 = 517 → Day 30 shortage = 517

Q95: Write a Python function that takes a filename and prints its first 10 lines.
✎ **Asked in:** BCC AP 2021

```
from itertools import islice
def head(filename, n=10):
    with open(filename) as f:
        for line in islice(f, n):
            print(line, end="")
head("notes.txt")
```

12-line file → prints l1 … l10

> **Exam trap:** The printed answer is Python 2 (`xrange`, `print head`), not a function, and crashes on a file shorter than 10 lines; `islice` stops safely.

34 Student records from a file: totals, grades, toppers

Q96: student.txt holds records: StudentID Name Marks1 Marks2 Marks3 Marks4. Write a program that (a) 3 calculates total and average marks; (b) 3 assigns grades: ≥ 80 A+, ≥ 70 A, ≥ 60 B, ≥ 50 C, < 50 F; (c) 3 displays the top three students; (d) 3 displays the subject with the highest average.
✎ **Asked in:** ICB Securities AP 2026(Aptitude)

```
struct st { int id; char name[20]; int m[4]; int total; double avg; } s[50], t;
int n = 0;  double sub[4] = {0};
FILE *fp = fopen("student.txt", "r");
if (!fp) return 1;
while (n < 50 && fscanf(fp, "%d %19s %d %d %d %d", &s[n].id, s[n].name,
                        &s[n].m[0], &s[n].m[1], &s[n].m[2], &s[n].m[3]) == 6) {
    s[n].total = 0;
    for (int k = 0; k < 4; k++) { s[n].total += s[n].m[k]; sub[k] += s[n].m[k]; }
```

```
    s[n].avg = s[n].total / 4.0;
    n++;
}
fclose(fp);
for (int i = 0; i < n; i++) {                                   /* (a), (b) */
    double a = s[i].avg;
    const char *g = a >= 80 ? "A+" : a >= 70 ? "A" : a >= 60 ? "B" : a >= 50 ? "C" : "F";
    printf("%d %-6s total %d avg %.2f grade %s\n", s[i].id, s[i].name, s[i].total, a, g);
}
for (int i = 0; i < n - 1; i++)                                 /* (c) sort by total */
    for (int j = 0; j < n - 1 - i; j++)
        if (s[j].total < s[j + 1].total) { t = s[j]; s[j] = s[j + 1]; s[j + 1] = t; }
printf("Top 3: %s, %s, %s\n", s[0].name, s[1].name, s[2].name);
int best = 0;                                                     /* (d) */
for (int k = 1; k < 4; k++) if (sub[k] > sub[best]) best = k;
printf("Highest-average subject: Marks%d (%.2f)\n", best + 1, sub[best] / n);
```

→ 101 Asif total 325 avg 81.25 grade A+ · 102 Mitu 254 63.50 B · 103 Rony 354 88.50 A+ · 104 Tania 185 46.25 F · 105 Sami 295 73.75 A · Top 3: Rony, Asif, Sami · Highest-average subject: Marks2 (72.40)

Marks are given for each part: read with `fscanf` until it fails (stop on a bad line), keep subject totals while reading, bubble-sort a copy by total for the toppers, and grade on the **average**.

> **Q97:** A Python program / a program using a Teletalk number 015… / "C programme output determination" — the tasks were not recalled in any source.
> ✎ **Asked in:** PGCB SAE(Computer) 2020 · Teletalk Executive(Technical) 2015 · NHA AP 2019

Prepare with the general programs on these sheets (loops, digits, strings) and the output drills that follow; for a phone-number task, treat it as a digit/string problem (validate 11 digits starting with 015, count or sum the digits).

35 Output questions: if, logic, shifts, division

Every output below was produced by compiling the exam's code (typos fixed as noted). Method: write the variables in a row, update them statement by statement, and remember **short-circuit**, **integer division** and **unsigned conversion**.

Code (as asked)	Output	Why	Asked in
`a = 85; if(a>=90) printf("A"); if(a>=80) printf("B"); if(a>=70) printf("C"); if(a>=60) printf("D"); else printf("F");`	**BCD**	four separate `if`s; the `else` belongs to the last one only	Petrobangla AM(CSE) 2024
`unsigned a = 3, b = 9; printf("a << 1 = %d ", a<<1); printf("b << 1 = %d", b<<1);`	**a << 1 = 6 b << 1 = 18**	left shift by 1 = × 2	WZPGCL AE(CSE) 2023
`a = -1, b = -a; x = (a>0)&&(b<0) || (a<0&&(b>0)); y = (a<=0) || (b>=0)&&(a>=0 || (b<=0)); printf("%d", x==y);`	**1**	x = 0 || 1 = 1; y = 1 (short-circuit). Read `a>==0` as `a>=0`, `%/d` as `%d`	DMTCL AE(ICT) 2023
`i=-3, j=1, k=0; m = ++i || ++j && ++k; printf("%d,%d,%d, %d", i,j,k,m);`	**-2,1,0, 1**	++i = −2 is true, so `||` skips the right side: j, k unchanged	DPDC JAM(ICT) 2020
`i=-20, j=-1, k=2; m = ++i && ++j && ++k;` print i, j, k, m	**-19, 0, 2, 0**	++j = 0 is false: `&&` stops, k stays 2	BCC AP 2017
`a=0, b=1, c=2; *((a) ? &b : &a) = a ? b : c;` print a, b, c	**2, 1, 2**	a = 0 picks `&a` on the left and c on the right: a = 2	BCC AP 2017
`unsigned int i = 10; if (i > -1) cout << "Yes"; else cout << "No";`	**No**	−1 is converted to unsigned = 4294967295, so 10 > it is false	BUET M.Sc. April 2024
`x=3, y=2; if (x==3) y=2; else y=3; cout << x << " " << y;`	**(b) 3 2**	x == 3, so y = 2	BB AD(ICT) 2025
`calc(x)`: ≥50 → 50, ≤20 → 20, else 35; returns 2x. `y=25; z = calc(y);` print y, z	**(c) y = 25, z = 70**	y is passed by value; 25 → 35 → 70	BUET M.Sc. April 2021
`i=5; l = i / -2; k = i % -2; printf("%d %d", l, k); x = 5 % 1;`	**(a) -2 1** then 0	division truncates toward 0; the remainder takes the sign of the dividend	BUET IICT April 2021
pseudocode: a=15, b=7; a = a mod (a−3); b = b mod (b−3); a = a mod 1; b = b mod 1; print a+b	**(c) 0**	15 mod 12 = 3, 7 mod 4 = 3; anything mod 1 = 0	BUET IICT April 2021
`printf("%c", 100);`	**(c) d**	ASCII 100 = 'd' ('a' = 97)	DoICT ANE 2019

> **Memory trick:** **&& stops at the first false, || stops at the first true**: cross out everything after the deciding operand before you trace.

> **Exam trap:** Petrobangla asked the a = 85 question three times (two copies lost the code): the answer is **BCD**, not "B", because the `if`s are not chained with `else`.

36 Increment side effects, macros, static, XOR

Code (as asked)	Output	Why	Asked in
`a = 5, b = 18, c = 27; a = b++ + ++c; b = a++ - c--; c = a++ + b--;` values of a, b, c?	**a = 48, b = 17, c = 65**	a = 18 + 28 = 46 (b 19, c 28); b = 46 − 28 = 18 (a 47, c 27); c = 47 + 18 = 65 (a 48, b 17)	46th BCS CSE
`#define MAX(A,B) ((A)>(B)) ? A:B` · `i=1, j=2; val = MAX(++i, ++j);`	**4**	expands to `((++i)>(++j)) ? ++i : ++j`: 2 > 3 false, then ++j again → 4	DESCO AE(CSE) 2016
`int sum(int v){ static int r = 0; r += v; return r; }` · `for(i=1;i<10;i++) value = sum(i);`	**45**	**static** keeps r between calls: 1 + 2 + … + 9	DESCO AE(CSE) 2016
`register static int i = 10; i = 11; printf("%d", i);`	**(b) compile-time error**	a variable can have only one storage class	BUET IICT April 2021
`a=10, b=20; a ^= b; b ^= a; a ^= b; printf("%d %d", a, b);`	**(c) 20 10**	XOR swap	Combined Bank AHE/AE(IT) 2021
`a=10, b=25; a = b++ + a++; b = ++b + ++a; printf("%d%d n", a, b);`	**3663 n** (expected)	a = 25 + 10 = 35, b = 26; then b = 27 + 36 = 63. `a = … a++` is **undefined**; clang prints 3663 n. The n is printed literally (the \ is missing)	DESCO AE(IT) 2022
`a = 1; printf("%d%d%d", ++a, a, a++);`	**undefined**	a is changed twice with no sequence point between arguments: clang prints **222**; the key (gcc, right to left) expects **331**	NWPGCL JAM(ICT) 2022
`i=4; printf("%d %d", ++i, i++); printf("%d", i++);` (a) 4 5 6 (b) 5 7 8 (c) 6 4 6 (d) 4 5 7	**undefined**; key (c)	same fault: gcc evaluates right to left → "6 4" then 6 → **(c)**; clang prints "5 5" then 6	Combined Bank AHE/AE(IT) 2021

Sequence point: a point where all earlier side effects are finished: the end of a full expression (`;`), `&&`, `||`, `?:`, the comma *operator*, and a function call (after its arguments). Changing a variable **twice** between two sequence points — or reading it while changing it — is **undefined behaviour**: the C standard allows any result.

Exam trap: Bangladeshi keys for `printf("%d %d", ++i, i++)` are built on gcc's right-to-left order. In the written exam write: "**undefined behaviour** (argument order unspecified); with gcc the output is …", and give the key's value.

Memory trick: A **macro is text copy-paste**: write the expansion out in full before counting the ++.

37 Loops: trace tables and endless loops

Q98: `sum = 0; for (i = -1, p = 0; i <= 10; i = i + 2) { p = i * i; if (i > 5) break; sum = sum + (i + p); printf("i=%d, p=%d, sum=%d\n", i, p, sum); } printf("Outside Loop, i=%d, p=%d, sum=%d", i, p, sum);`
✎ **Asked in:** BGDCL AE(CSE) 2021

i	p = i²	i > 5?	sum
−1	1	no	0
1	1	no	2
3	9	no	14
5	25	no	44
7	49	**yes: break**	44

i=-1, p=1, sum=0
i=1, p=1, sum=2
i=3, p=9, sum=14
i=5, p=25, sum=44
Outside Loop, i=7, p=49, sum=44

Code (as asked)	Output	Why	Asked in
(a) `n = 10; for(;;) printf("%d", n);` or (b) `i = 2, j = 2; while (i ? --i : j++) printf("%d", i);`	(a) **10101010…** forever · (b) **1**	(a) `for(;;)` has no condition. (b) i = 2 → --i = 1 true, print 1; then --i = 0 false: stop	PSB AP 2018
(a) `char str[120] = "Digital Bangladesh"; n = strlen(str); str[4] = '\0'; printf("%s", str);` or (b) `for(i=0;i<5;i++){ if(i==3) continue; printf("%d\n", i); }`	(a) **Digi** · (b) **0 1 2 4**	(a) '\0' at index 4 ends the string. (b) `continue` skips 3	PSB Programmer 2018
`#define N 7` · `char str[] = "abpqx "; for(i=0;i<N-2;i++) if(i%2) printf("%c", str[i]++); else printf("%c", str[i]--);`	**abpqx**	postfix ++/-- print first, change after; i = 0 … 4 (`Char` read as `char`)	BTCL AE(CSE) 2017
(i) Java `Integer a = new Integer(33), b = new Integer(33); if (a == b) … println(a + b + 'a');` (ii) `i = 7; for(; i != 0; i--) printf("x=%d", i--);` (iii) `j = 1; while (j++ < 50) println(j + "");` (iv) `unsigned a = .1, b = 2; printf("%d", a << a + b);`	(i) **163** · (ii) **x=7x=5x=3x=1x=-1…** forever · (iii) **2 … 50** · (iv) **0**	(i) two objects: == false, but both branches print 33 + 33 + 97. (ii) i falls by 2 a pass and jumps over 0. (iii) 49 lines (add the missing)). (iv) a = 0	Sonali Bank ADA 2020

Q99: Find the output and analyse the time and space complexity: `n; first = 0; secnd = 1; if (n >= 1) cout << first; else if (n <= 2) cout << secnd; for (i = 2; i < n; i++) { next = first + secnd; cout << next; first = secnd; secnd = next; }`
✎ **Asked in:** Rupali Bank ANA 2023

For n = 5 it prints 0 1 2 3 — Fibonacci with the **second 1 missing**: the `else if` never prints `secnd` when n ≥ 1 (fix: print 0 and 1 for n ≥ 2). Time **O(n)** (one loop), space **O(1)** (three variables). The printed `next + first + secnd;` is read as `next = first + secnd;`.

38 Pointers, arrays, recursion, strings

Code (as asked)	Output	Why	Asked in
`int arr[5] = {1,2,3,4,5}; int *p = (arr + 2); printf("%d", p[-1]);`	**(ক) 2**	p points at arr[2]; p[−1] = arr[1]	49th BCS CSE (2025)
`int fun(int *p){ *p = *p + 10; return *p; }` · `x = 5; cout << fun(&x);`	**(a) 15**	changed through the pointer	BB AD(ICT) 2025
`int a[] = {5,3,2,1,4}; int b[] = {66,55,30,20,45}; printf("%d", b[a[2]]);`	**30**	a[2] = 2, b[2] = 30 (printed `arr[a]` read as `a[]`)	BSCPLC AM(Engineering) 2024
`int k = 7; int *p = &k; int **m = &p; printf("%d%d%d", k, *p, **m);`	**(c) 7 7 7** (printed 777)	all three reach k	BUET IICT April 2021
`int a = 100; int *p = &a + 2; *p = 22; printf("%d", a);`	**undefined**; key (a) 100	writes outside a: a itself stays 100 if the program survives; our run crashed	Combined Bank AHE/AE(IT) 2021
`int (*fp)(int, int); fp = mul;` (mul takes 3 ints) `printf("…%d", fp(2, 3));`	**(b) the code is incorrect**	incompatible function-pointer type; clang refuses, gcc warns and the call is UB	BUET IICT April 2021
`bitwise(2147483649)` swaps bit i with bit 31−i (mask1 from the right, mask2 from the left)	**(d) 2147483649**	2147483649 = bits 31 and 0 set: its mirror image is itself	BUET M.Sc. April 2021
`int function(int x[], int i){ int s = x[i]; if(i>0) s += function(x, i-1); printf("%d", s); return s; }` · `function(y, 2)`, y = {1,3,2,8}	**146**	deepest call prints first: 1, then 1+3 = 4, then 4+2 = 6	BCC AP 2021
`fun(i){ if(i<=0) return; printf("%d, ", i--); fun(--i); printf("%d, ", i); }` · `fun(5)`	**5, 3, 1, -1, 1, 3,**	each call prints i, recurses with i − 2, then prints its own i (now i − 2) on the way back	Combined Bank AP 2024
(i) `foo(x, y){ if(x==0) return y; return foo(x-1, x+y); }` · `foo(3, 4)` (ii) `foo(char *a){ if(*a && *a != ' '){ foo(a+1); putchar(*a); } }` · `"COVID-19"`	(i) **10** · (ii) **91-DIVOC**	(i) (3,4)→(2,7)→(1,9)→(0,10). (ii) print after the call = reverse	BUET AP 2025
Java: `static int x = 5; fun(n){ if(n<=1) return 1; x = x + 2; return fun(n-1) + x; }` · `fun(3)`	**19**	x = 7, then fun(2): x = 9, returns 1 + 9 = 10; back in fun(3): 10 + x (now **9**) = 19	Combined Bank SO(IT) 2026
`string cars[4] = {"Volvo","BMW","Ford","Mazda"}; cout << cars[3]; cout << cars[2];`	**(c) MazdaFord**	no space or newline between them	BUET IICT April 2021
`string S = "Wasa Authority", s = S, R = S;` loop: even i → `S[i] = toupper`, `s[i] = tolower`; print s and R; `return 0;` inside the loop	**s: wasa Authority r: Wasa Authority**	the `return` ends main after the first pass (i = 0): only s[0] became 'w'	Dhaka WASA AME(Network) 2025

Memory trick: Recursion output: **print before the call → top-down order; print after the call → reverse order**.

39 How to answer any output question

Q100: Explain the while-loop of the given program step by step for inputs 1221 and 3456 and write the output (the program was not printed; it is the reverse-and-compare palindrome loop).
✎ **Asked in:** BAPEX AGM(ICT) 2023

n	n % 10	rev
1221	1	1
122	2	12
12	2	122
1	1	**1221**

`while (n != 0) { rev = rev * 10 + n % 10; n /= 10; }` then compare rev with the original: 1221 → 1221, palindrome; 3456 → 6543, not a palindrome.

The papers below set an output question whose code was **not preserved** in any copy. They test the same traps as the four previous sheets; drill those.

- "After compilation and execution, what will be the output?" — see the `++i || ++j && ++k` family. ✎ DPDC JAM(ICT) 2020
- "Write down the output of this program." ✎ DESCO AE(IT) 2022
- "Program analysing and writing output." ✎ DESCO AE(CSE) 2016
- "Find the output of the following program." ✎ Sonali Bank Officer(IT) 2021
- "Find output" (code not on the page). ✎ NWPGCL AE(CSE)
- "C programming output problem" (incomplete question). ✎ Teletalk AM(IT) 2023

- "Given Python code, find its output with complexity": count the loops — one loop over n is O(n), nested loops O(n²), halving O(log n). ✎ RAKUB ANSE 2023
- "String compare code in C: write the output": `strcmp(a, b)` returns **0** if equal, a **negative** value if a comes first, **positive** if after; `a == b` compares addresses, not text. ✎ BUET M.Sc. October 2018
- "Will it compile? If so, what is the output?": check declarations, types, missing `;`, storage-class clashes (`register static`), pointer types, then trace. ✎ BUET M.Sc. October 2018
- "C code with pointers: is there a problem? If not, the output": look for an uninitialised pointer, returning the address of a local, out-of-bounds index, missing `free`, double `free`. ✎ BUET M.Sc. May 2019

The ten traps examiners set

1. **Short-circuit**: `&&` / `||` skip the right side (DPDC, BCC).
2. **Separate ifs** vs if-else chain (Petrobangla BCD).
3. **Integer division**, truncation toward 0, sign of `%` (BUET IICT).
4. **Unsigned vs −1** comparison (BUET 2024).
5. **Pre vs post** ++, and **undefined behaviour** when a variable changes twice in one expression.
6. **Macro expansion** evaluates arguments again (DESCO MAX).
7. **static** locals keep their value (DESCO sum = 45).
8. **Loop that skips its exit** (i -= 2 past 0; `for(;;)`).
9. **'\0' cuts a string**; `%c` of a number prints its ASCII character.
10. **Recursion print order**: before the call vs after it.

★ Night before the exam

1. **Call by value** = copy, caller unchanged · **by reference** = address (`int *x`, `f(&a)`), caller changes (5 exams).
2. **Recursion** = base case + recursive case; factorial `n * fact(n-1)`, Fibonacci, GCD `gcd(b, a % b)`; one stack frame per call.
3. **Compiler** whole program, fast, object file · **interpreter** line by line, stops at the first error · **linker** joins libraries · **loader** loads to RAM.
4. **Array vs structure**: same type by index vs mixed types by name · **struct vs union**: own space vs shared (size of the largest).
5. **malloc** garbage, one argument · **calloc** zeroes, two arguments · always `free`.
6. **while** tests first (0 or more) · **do-while** exit-controlled (at least once) · **break** leaves, **continue** skips.
7. **Digits**: % 10 peels, / 10 drops, × 10 + pushes · **LCM** = a × b / GCD · primes to √n.
8. **Output traps**: short-circuit, unsigned −1, integer division, macro re-evaluation, static, UB with ++.

✓ Practice MCQs (all asked; answers at the bottom)

1. Exit-controlled loop: (a) for (b) while (c) do-while (d) none ✎ 49th BCS CSE (2025)

2. Keep a variable's value between calls: (a) auto (b) extern (c) static (d) register ✎ 49th BCS CSE (2025)

3. Array of four pointers to float: (a) `float Arr[4]` (b) `(float) Arr[4]` (c) `float *Arr[4]` (d) `Ptr (float) Arr[4]` ✎ 49th BCS ICT (2025)

4. Size of `int array[15]` (4-byte int): (a) 15 (b) 19 (c) 11 (d) 60 ✎ BPSC AP 2022

5. Passing an array passes: (a) all elements (b) the first element (c) the address of the first element (d) the last address ✎ BPSC AME 2019

6. `(x < 0) ? 10 : 15`, x = 5: (a) 10 (b) 15 (c) 0 (d) 1 ✎ NWPGCL JAM(ICT) 2022

7. Converts a whole program to machine language: (a) interpreter (b) converter (c) simulator (d) compiler ✎ BPSC AP 2022

8. `printf("%c", 100);` (a) 100 (b) one hundred (c) d (d) 0 ✎ DoICT ANE 2019

Answers: 1 c · 2 c · 3 c · 4 d · 5 c · 6 b · 7 d · 8 c

Memory trick: Every C answer: **definition → table → program → sample output**. Programs without an output line lose marks.

Exam trap: Write `int main(void)` and `return 0;`, not `void main` and `conio.h`: several printed solutions lose marks for that.